University of Mannheim

Master Thesis

Quasi-Monte Carlo Methods and Applications

Author:
Janik V. Hrubant

Supervisors:
Prof. Dr. Andreas Neuenkirch

A thesis submitted in fulfillment of the requirements for the degree of Master of Science

in the

Research Group Name
Department or School Name

September 2, 2025

Declaration of Authorship

English Version. I hereby declare that I have completed this thesis independently and without any unauthorized assistance. I further confirm that neither this thesis nor any part of it has been submitted by me or by others as part of any other academic assessment. Literal or paraphrased quotations from other sources—whether in print or electronic form—are clearly marked as such. All secondary literature and other sources used are properly cited and listed in the bibliography. This also applies to graphical representations, images, and all online sources. I furthermore agree that my thesis may be submitted and stored in anonymized electronic form for the purpose of plagiarism detection. I am aware that the thesis may not be evaluated if this declaration is not submitted.

German Version. Hiermit versichere ich, dass diese Arbeit von mir persönlich verfasst wurde und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen. Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn diese Erklärung nicht erteilt wird.

Place, Date

Signature

Abstract

Here comes the Abstract or Introduction of the thesis.

Contents

Acronyms

RITA Rational Inverse Transform with Aliasing

CDF Cumulative Distribution Function

QMC Quasi-Monte Carlo

MC Monte Carlo

HU Hounsfield Unit

CT Computed Tomography

CBCT Cone-Beam Computed Tomography

FFD Forced Fixed Detection

DNN Deep Neural Network

ODE Ordinary Differential Equation

SGD Stochastic Gradient Descent

Adam Adaptive Moment Estimation

CDF Cumulative Distribution Function

Part I

Fundamentals of Quasi-Monte Carlo Methods

Chapter 1

Monte Carlo and Quasi-Monte Carlo Integration

1.1 Motivation and Problem Setting

The numerical evaluation of high-dimensional integrals is a fundamental task in modern applied mathematics and scientific computing. Applications range from Bayesian inference and financial mathematics to machine learning and computational physics. In particular, contemporary domains such as deep neural network training and medical imaging often require the estimation of integrals of the form

$$I(f) = \int_{[0,1]^s} f(x)\,dx, \tag{1.1}$$

where s denotes the dimensionality of the problem and f is a function, where the integral is expensive or impractical to evaluate analytically. Throughout this thesis, we will limit our discussion on functions of the type $f : \mathbb{R}^s \to \mathbb{R}$.

One of the most widely used approaches to compute such integrals is the classical Monte Carlo (MC) method, which estimates the integral based on averages over randomly sampled points $\{X_n\}_{n=0}^{N-1}$

$$\frac{1}{N}\sum_{n=0}^{N-1} f(X_n) \approx \int_{[0,1]^s} f(x)\,dx.$$

The primary appeal of MC integration lies in its dimension-independent convergence rate and minimal assumptions on the integrand. However, its asymptotic error rate of $\mathcal{O}(N^{-1/2})$ [14] limits its efficiency — especially when high precision is required or function evaluations are computationally expensive.

Quasi-Monte Carlo (QMC) methods offer an alternative paradigm: instead of random samples, they employ deterministic sequences — so-called low-discrepancy sequences — that aim to fill the integration domain more uniformly. This structured sampling allows for faster convergence under certain smoothness conditions and forms the basis for many state-of-the-art techniques in high-dimensional numerical integration.

The first part of this thesis is structured into three chapters. Chapter 1 introduces the concepts of MC and QMC methods to estimate the integral from (1.1). Chapter 2 provides a formal introduction to discrepancy theory and some constructions of low-discrepancy sequences. In Chapter 3, the concepts of variation and error bounds such as the *Koksma-Hlawka inequality* are introduced. The subsequent Part II of this thesis focuses on the application of QMC methods for training of neural networks, while Part III applies QMC techniques to photon transport simulation in Computed Tomography.

1.2 Monte Carlo Integration: Principle and Convergence

The classical MC method is a probabilistic approach to numerically evaluate the integral $I(f)$ as defined in (1.1). The goal is to approximate this integral by averaging function evaluations at randomly sampled points in the integration domain $[0,1]^s$.

Definition 1.2.1 (Monte Carlo Estimator)**.**
Let $f \in L^2([0,1]^s)$ and let $X_0, \dots, X_{N-1} \overset{\text{i.i.d.}}{\sim} \mathcal{U}([0,1]^s)$ be independent and identically distributed drawn random samples. Then the Monte Carlo estimator of the integral I from (1.1) is given by

$$I_N^{\text{MC}}(f) := \frac{1}{N} \sum_{n=0}^{N-1} f(X_n). \tag{1.2}$$

Remark 1.2.2.
Let $f \in L^1([0,1]^s)$. Then by the Strong Law of Large Numbers, the Monte Carlo Estimator from (1.2) converges almost surely to the integral $I(f)$ from (1.1) as the number of samples N tends to infinity:

$$\mathbb{P}\left(\lim_{N\to\infty} I_N^{\text{MC}}(f) = \int_{[0,1]^s} f(\mathbf{x})\, \mathrm{d}\mathbf{x} \right) = 1, \tag{1.3}$$

i.e. $I_N^{\text{MC}}(f) \xrightarrow{\text{a.s.}} \int_{[0,1]^s} f(\mathbf{x})\, \mathrm{d}\mathbf{x}$ for $N \to \infty$.

Beyond the almost sure convergence by the Strong Law of Large Numbers, the MC estimator has a well-known convergence rate as stated in [14, Section 1.2].

Theorem 1.2.3 (Monte Carlo Convergence Rate)**.**
Let $f \in L^2([0,1]^s)$. Then

$$\mathbb{E}\left[\left| I_N^{\text{MC}}(f) - I(f) \right| \right] \leq \frac{\sigma[f]}{\sqrt{N}}, \tag{1.4}$$

where $\sigma[f] := \sqrt{\text{Var}[f]}$.

Sketch of Proof.
By independence,

$$\mathrm{Var}\Big(I_N^{\mathrm{MC}}(f)\Big) = \frac{\mathrm{Var}[f]}{N}.$$

With $X = I_N^{\mathrm{MC}}(f) - I(f)$, Jensen's inequality for the concave $\phi(t) = \sqrt{t}$ gives

$$\mathbb{E}[|X|] = \mathbb{E}[\phi(X^2)] \leq \phi(\mathbb{E}[X^2]) = \sqrt{\mathrm{Var}(X)}.$$

This yields the stated $O(N^{-1/2})$ bound. □

Despite its robustness and simplicity, MC integration has several well-known limitations:

- The convergence rate is relatively slow, especially for smooth integrands.
- Error bounds are probabilistic rather than deterministic.
- The method does not exploit structural properties of the integrand, such as smoothness or sparsity.

These limitations arise due to the imperfect uniformity of random sample points in the hypercube $[0,1]^s$, a phenomenon that motivates the introduction of the notions of uniformity and discrepancy in the next section.

1.3 Uniformity Concepts

The efficiency of QMC methods hinges on how well the employed point sets "fill" the integration domain $[0,1]^s$. This motivates the need for a precise mathematical understanding of *uniformity*. The present section introduces two key concepts in this regard — *uniform distribution modulo one* and *equidistribution* — and clarifies their relevance to numerical integration.

1.3.1 Uniform Distribution Modulo One

In QMC methods, the notion of *uniform distribution modulo one* provides a rigorous criterion for how evenly a sequence covers the unit cube. This concept forms the foundation for analyzing the convergence behavior of QMC estimators.

Definition 1.3.1 (Uniform Distribution Modulo One)**.**
Let $(\boldsymbol{x}_n)_{n\in\mathbb{N}_0} \subset [0,1]^s$ be a sequence of sample points. The sequence is said to be *uniformly distributed modulo one* (u.d. mod 1) if for every axis-aligned box $[\boldsymbol{a},\boldsymbol{b}) \subset [0,1]^s$, the proportion of points falling into this box converges to its Lebesgue measure:

$$\lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} \chi_{[\boldsymbol{a},\boldsymbol{b})}(\boldsymbol{x}_n) = \lambda_s([\boldsymbol{a},\boldsymbol{b}))\,,$$

where $\chi_{[\boldsymbol{a},\boldsymbol{b})}$ is the indicator function of the set $[\boldsymbol{a},\boldsymbol{b})$ and λ_s denotes the s-dimensional Lebesgue measure.

Here $[\boldsymbol{a},\boldsymbol{b})$ denotes the half-open hypercube in $\mathbb{R}^s$ defined by

$$[\boldsymbol{a},\boldsymbol{b}) = \{\boldsymbol{x} \in \mathbb{R}^s \mid a_i \leq x_i < b_i \text{ for } i = 1,\dots,s\}.$$

This definition emphasizes asymptotic spatial coverage: in the limit, every subregion of the domain is sampled proportionally to its volume. Importantly, this property is purely deterministic and does not rely on any probabilistic assumptions — in contrast to Monte Carlo methods, which only guarantee uniformity in expectation.

In contrast, when speaking of a random sample $\{X_1,\dots,X_N\} \subset [0,1]^s$ drawn independent and identically distributed (i.i.d.) from the uniform distribution, we refer to a probabilistic concept of uniformity: each point is independently drawn according to the uniform measure, but the empirical distribution may not be uniformly spread in finite samples. Thus, equidistribution refers to the ideal uniform coverage that we seek deterministically, while uniform random sampling only ensures this behavior in expectation or with high probability. In Figure 1.1, a randomly sampled set of 300 points in $[0,1]^2$ is compared to the first 300 sequence elements from a QMC sequence. The latter sequence is u.d. mod 1.

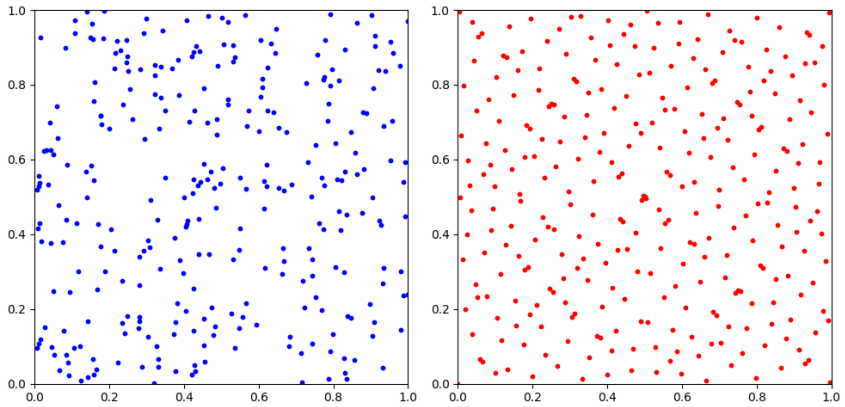


Figure 1.1: Comparison of 300 randomly drawn sample points in $[0,1]^2$ using standard MC (left) and the first 300 elements of the Sobol' sequence in 2 dimension.

Remark 1.3.2.
Uniform distribution modulo one is a necessary condition for the convergence of QMC estimators. If a point sequence is not u.d. mod 1, then there exists at least one Riemann-integrable function for which the sample average

$$\frac{1}{N}\sum_{n=1}^{N} f(x_n)$$

fails to converge to the integral. [14, Section 2.1]

The following result, known as Weyl's criterion, provides a necessary and sufficient condition for uniform distribution in terms of exponential sums.

Theorem 1.3.3 (Weyl's Criterion)**.**
A sequence $(\boldsymbol{x}_n)_{n\geq 0} \subset [0,1)^s$ is uniformly distributed modulo one if and only if, for all nonzero $\boldsymbol{h} \in \mathbb{Z}^s \setminus \{\mathbf{0}\}$, we have

$$\lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} \exp(2\pi i\, \boldsymbol{h}\cdot \boldsymbol{x}_n) = 0.$$

A proof can be found in [14, Section 2.1].

Example 1.3.4.
Let $\alpha \in \mathbb{R}$ be irrational. Then the Kronecker sequence $(n\alpha \bmod 1)_{n\geq 0}$ is uniformly distributed in $[0,1]$. This is a consequence of Weyl's criterion and illustrates the existence of simple, deterministic sequences with excellent uniformity properties.

Chapter 2

Discrepancy Theory and Low-Discrepancy Sequences

2.1 Measuring Uniformity: Discrepancy

One of the cornerstones of QMC theory is the concept of *discrepancy*, which quantifies how uniformly a finite point set samples the s-dimensional unit cube $[0,1]^s$. While uniform distribution modulo one describes the asymptotic behavior of infinite sequences, discrepancy provides a finite-sample measure of deviation from perfect uniformity. Low-discrepancy sequences are the foundation of QMC methods because they minimize this deviation and thus yield more accurate numerical integration.

This section introduces formal definitions, geometric intuition and empirical insights into discrepancy measures. It lays the theoretical groundwork for understanding how the structure of point sets affects QMC integration performance.

2.1.1 Definition of Discrepancy and Star Discrepancy

First, we define the more general *extreme discrepancy*:

Definition 2.1.1 (Extreme Discrepancy)**.**
Let $\mathcal{P} \subset [0,1)^s$ with $|\mathcal{P}| = N$ be a finite point set. Then the extreme discrepancy $D_N(\mathcal{P})$ is defined as

$$D_N(\mathcal{P}) = \sup_{\substack{\boldsymbol{a},\boldsymbol{b}\in[0,1]^s \\ \boldsymbol{a}\leq\boldsymbol{b}}} \left| \frac{A([\boldsymbol{a},\boldsymbol{b}),\mathcal{P})}{N} - \lambda_s([\boldsymbol{a},\boldsymbol{b})] \right|.$$

Here $A([\boldsymbol{a},\boldsymbol{b}),\mathcal{P})$ denotes the number of points in $\mathcal{P}$ that fall into the box $[\boldsymbol{a},\boldsymbol{b})$, and $\lambda_s([\boldsymbol{a},\boldsymbol{b})]$ is the Lebesgue measure of that box, given by $\prod_{i=1}^{s}(b_i - a_i)$.

In practice, a common and slightly more tractable variant is the *star discrepancy*, which restricts the test boxes to be anchored at the origin.

Definition 2.1.2 (Star Discrepancy)**.**
Let $\mathcal{P} \subset [0,1)^s$ with $|\mathcal{P}| = N$ be a finite point set. Then the star discrepancy D_N^* is defined as

$$D_N^*(\mathcal{P}) = \sup_{\boldsymbol{t} \in [0,1]^s} \left| \frac{A([\mathbf{0}, \boldsymbol{t}), \mathcal{P}, N)}{N} - \lambda_s([\mathbf{0}, \boldsymbol{t})] \right|.$$

This form is used in the Koksma–Hlawka inequality (Theorem 3.2.1) and serves as a central measure in QMC analysis. Note that both discrepancy definitions are deterministic and depend solely on the point configuration.

Remark 2.1.3.
A low star discrepancy implies that the empirical distribution of the points approximates the uniform distribution well over all axis-aligned subrectangles anchored at the origin.

2.1.2 Geometric Interpretation and Examples

To build geometric intuition, consider the 1-dimensional case. Given N sample points $x_0, \ldots, x_{N-1} \in [0,1)$, we can visualize discrepancy as the maximum vertical deviation between the empirical distribution function

$$F_N(t) := \frac{1}{N} \sum_{n=0}^{N-1} \chi_{[0,t)}(x_n)$$

and the uniform cumulative distribution function $F(t) = t$. The discrepancy corresponds to the largest gap between $F_N(t)$ and $F(t)$.

In higher dimensions, the idea generalizes: the star discrepancy measures the maximal difference in the number of points falling into an axis-aligned box $[\mathbf{0}, \boldsymbol{t})$ versus the volume of that box. Intuitively, it quantifies whether the point set "overpopulates" or "underpopulates" certain regions.

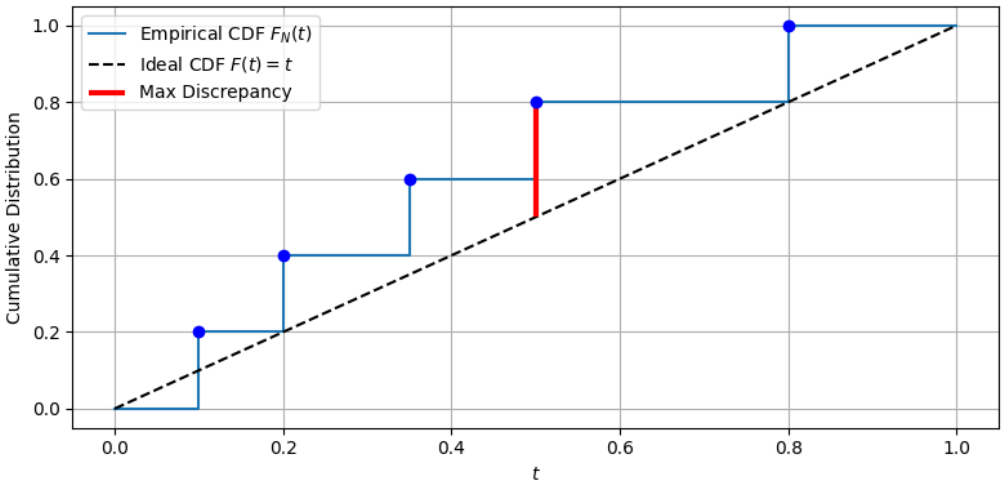


Figure 2.1: Visualization of 1D discrepancy: the maximum vertical distance between the empirical CDF $F_N(t)$ and the uniform CDF $F(t) = t$.

Example 2.1.4.
Let $x_n = \frac{n}{N}$ for $n = 0, \ldots, N-1$. Then $F_N(t)$ is a piecewise constant staircase function. With Theorem 2.4 from [32], one gets that the discrepancy is $\frac{1}{N}$. This is an optimal rate in 1D.

Remark 2.1.5.
Discrepancy provides a worst-case error metric over all subintervals. Thus, even a point set that appears visually well-distributed may have large discrepancy due to subtle gaps or clustering in certain regions.

2.1.3 Empirical Observation of Discrepancy in QMC

Discrepancy is not just a theoretical construct – its practical relevance becomes evident when comparing the behavior of QMC sequences with MC samples. In particular, structured point sets like Halton and Sobol' sequences as introduced in the next section, achieve much lower discrepancy than random samples of the same size.

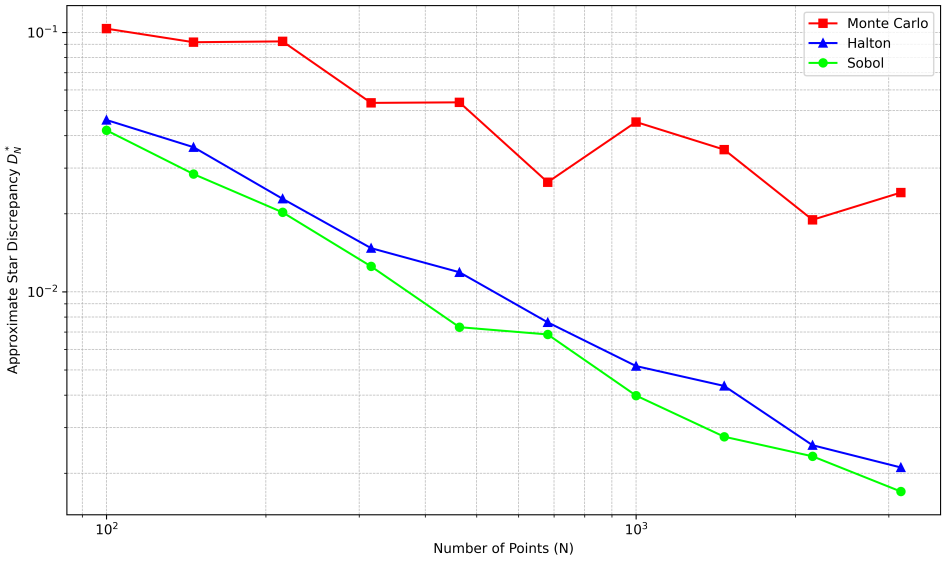


FIGURE 2.2: Comparison of discrepancy growth for Sobol', Halton and random (MC) point sets in 2D. Low-discrepancy sequences exhibit significantly slower discrepancy growth.

Remark 2.1.6.
The expected star discrepancy of i.i.d. Monte Carlo samples decreases at the rate $\mathcal{O}(N^{-1/2})$, whereas for low-discrepancy sequences provable upper bounds of order $\mathcal{O}((\log N)^s/N)$ exist. [14, Section 2.2]

2.2 Low-Discrepancy Sequences

Now we will introduce the concept of low-discrepancy sequences and analyse their dimensional performance properties. We start by defining the term of low-discrepancy sequences, which are designed to fill the unit cube $[0,1)^s$ as uniformly as possible.

Definition 2.2.1 (Low-Discrepancy Sequence).
A point set $\mathcal{P} = \{\boldsymbol{x}_0, \dots, \boldsymbol{x}_{N-1}\} \subset [0,1)^s$ with $|\mathcal{P}| = N$ is called a low-discrepancy point set if its star discrepancy satisfies

$$D_N^*(\mathcal{P}) = \mathcal{O}\left(\frac{(\log N)^s}{N}\right). \tag{2.1}$$

A sequence $\mathcal{S} = (x_n)_{n\in\mathbb{N}}$ in $[0,1)^s$ is called a low-discrepancy sequence if its star discrepancy satisfies

$$D_N^*(\mathcal{S}) = \mathcal{O}\left(\frac{(\log N)^s}{N}\right) \tag{2.2}$$

for all $N \in \mathbb{N}$. Such sequences are also referred to as *Quasi-Monte Carlo sequences* or QMC sequences.

Remark 2.2.2.
Low-discrepancy sequences are constructed to minimize the star discrepancy. For dimension $s = 1$ and $s = 2$, it is known that (2.1) is an optimal bound. K. F. Roth proved in 1954 that for everz dimension $s \in \mathbb{N}$ there exists a constant $c_s > 0$ such that for every point set $\mathcal{P} \subset [0,1)^s$ with $|\mathcal{P}| = N$ the lower bound

$$D_N(\mathcal{P}) \geq D_N^*(\mathcal{P}) \geq c_s \frac{(\log N)^{\frac{s-1}{2}}}{N}$$

holds [14, Theorem 2.24]. It is still an open problem whether the bound also holds with exponent $s-1$ instead of $\frac{s-1}{2}$ for $s \geq 3$. Nevertheless, since this is conjectured to be true, it has become common in literature to refer to sequences satisfying (2.2) as low-discrepancy sequences. The bound on the discrepancy is crucial for the convergence of QMC methods according to the Koksma-Hlawka inequality introduced in Chapter 3.

With the definition of a low-discrepancy sequence in place, we now get back to the MC estimator from 1.2.1 and define its equivalent QMC estimator:

Definition 2.2.3 (Quasi-Monte Carlo Estimator).
Let $f\colon [0,1]^s \to \mathbb{R}$ be a measurable function and let $\{\boldsymbol{x}_n\}_{n=1}^N \subset [0,1]^s$ be a QMC point set. Then the *Quasi-Monte Carlo estimator* for the integral

$$I(f) = \int_{[0,1]^s} f(\boldsymbol{x})\, d\boldsymbol{x}$$

is defined as

$$I_N^{\mathrm{QMC}}(f) := \frac{1}{N}\sum_{n=1}^{N} f(\boldsymbol{x}_n). \tag{2.3}$$

Remark 2.2.4.
Even though grid points as in Example 2.1.4 can achieve optimal discrepancy in 1D, they are not considered low-discrepancy sequences in the sense of the above definition, since they do not satisfy the asymptotic bound for all N. Furthermore, grids are not easily extensible – adding new points requires global recomputation

and destroys nesting.

Now that we have established the definitions of low-discrepancy sequences we will introduce two examples of low-discrepancy sequences that are widely used in practice: the Halton sequence and the Sobol' sequence.

2.2.1 The Halton Sequence

The Halton sequence is one of the earliest and most widely used constructions of low-discrepancy sequences in arbitrary dimensions.

The construction of the Halton sequence is based on the one-dimensional *Van der Corput sequence*, introduced by Johannes van der Corput in 1935. Therefore, we utilize the radical-inverse function $\phi_b(n)$, which maps an integer n to a real number in $[0,1)$ by reflecting its base-b representation about the decimal point. This function is defined as follows:

$$\phi_b(n) = \sum_{k=0}^{\infty} d_k b^{-k-1}, \quad \text{where } n = \sum_{k=0}^{\infty} d_k b^k \quad \text{and } d_k \in \{0,1,\dots,b-1\}.$$

Definition 2.2.5 (Van der Corput Sequence)**.**
The one-dimensional Van der Corput sequence in base b is defined as

$$\mathcal{S}_b = (\phi_b(n))_{n\in\mathbb{N}}.$$

John Halton generalized this idea to higher dimensions by combining multiple Van der Corput sequences in different bases in 1960 [9].

Definition 2.2.6 (Halton Sequence)**.**
Let $b_1,\dots,b_s$ be pairwise coprime integers greater than 1, typically chosen as the first s prime numbers. The n-th point $\boldsymbol{x}_n \in [0,1)^s$ of the s-dimensional Halton sequence is defined componentwise by

$$\boldsymbol{x}_n = \Big(\phi_{b_1}(n),\dots,\phi_{b_s}(n)\Big),$$

where $\phi_b(n)$ denotes the Van der Corput radical-inverse function in base b. The Halton sequence is then given by $\mathcal{S}_{b_1,\dots,b_s} = (x_n)_{n\in\mathbb{N}}$.

Example 2.2.7 (First Elements of the Halton Sequence in 2D).
Consider the two-dimensional Halton sequence based on the first two prime bases $b_1 = 2$ and $b_2 = 3$. The first three elements are obtained by computing the radical-inverse values $\phi_{b_1}(n)$ and $\phi_{b_2}(n)$ for $n = 1,2,3$:

- $n = 1$: $\boldsymbol{x}_1 = (\phi_2(1),\phi_3(1)) = \left(\frac{1}{2},\frac{1}{3}\right)$
- $n = 2$: $\boldsymbol{x}_2 = (\phi_2(2),\phi_3(2)) = \left(\frac{1}{4},\frac{2}{3}\right)$
- $n = 3$: $\boldsymbol{x}_3 = (\phi_2(3),\phi_3(3)) = \left(\frac{3}{4},\frac{1}{9}\right)$

This illustrates how the Halton sequence fills the unit square in a low-discrepancy manner even for small n.

Intuitively, this construction ensures that each component of the sequence explores the unit interval in a structured, non-redundant way. By combining several one-dimensional Van der Corput sequences in different coprime bases, the resulting multi-dimensional point set avoids regular grid-like patterns and achieves asymptotic uniformity.

As stated in [14, Section 2.4], the Halton sequence has star discrepancy of order

$$D_N^*(\{\boldsymbol{x}_0,\dots,\boldsymbol{x}_{N-1}\}) = \mathcal{O}\left(\frac{(\log N)^s}{N}\right),$$

making it a prototypical example of a low-discrepancy sequence. However, for larger dimensions, correlation effects between the different base components can lead to degraded uniformity and higher discrepancy. To mitigate this, variants are used in practice: leaping (taking only every k-th element of the sequence) and scrambling (applying randomized permutations to the digits in the radical-inverse function). Both reduce correlation effects and improve uniformity in higher dimensions.

Remark 2.2.8.
The Halton sequence is extensible in N and s, making it suitable for applications that require growing or adaptive point sets. However, its distribution properties in higher dimensions tend to be inferior compared to constructions such as the Sobol' sequence.

2.2.2 The Sobol' Sequence

Sobol' sequences are another class of low-discrepancy sequences that are widely used in QMC methods, particularly for high-dimensional problems. Constructed to fill the s-dimensional unit cube $[0,1)^s$ as uniformly as possible, Sobol' sequences are based on a recursive structure that allows for efficient generation and high-dimensional performance.

Therefore functions from the *ring of polynomials*

$$\mathbb{F}_2[x] = \{a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n \mid n \in \mathbb{N},\ a_i \in \mathbb{F}_2\}$$

with coefficients in the finite field $\mathbb{F}_2$ are necessary. $\mathbb{F}_2 = \{0,1\}$ denotes the finite field with two elements, equipped with addition and multiplication modulo 2. A primitive polynomial of degree m in $\mathbb{F}_2[x]$ is an irreducible polynomial whose roots generate all non-zero elements of the field extension $\mathbb{F}_{2^m}$.

Definition 2.2.9 (Sobol' Sequence Construction)**.**
Let $p_1,\dots,p_s \in \mathbb{F}_2[x]$ be primitive polynomials ordered by non-decreasing degree. Each polynomial p_j for dimension j is of the form

$$p_j(x) = x^{e_j} + a_{1,j}x^{e_j-1} + \cdots + a_{e_j-1,j}x + 1,$$

where $e_j \in \mathbb{N}$ and the coefficients $a_{k,j} \in \{0,1\}$.
Choose initial values $m_{1,j},\dots,m_{e_j,j}$ such that each $m_{k,j}$ is an odd integer and $1 \le m_{k,j} < 2^k$ for $1 \le k \le e_j$. For $k > e_j$, the values $m_{k,j}$ are computed recursively as:

$$\begin{aligned} m_{k,j} =& 2a_{1,j} \cdot m_{k-1,j} \oplus 2^2 a_{2,j} \cdot m_{k-2,j} \oplus \dots \\ & \oplus 2^{e_j-1} a_{e_j-1,j} \cdot m_{k-(e_j-1),j} \oplus 2^{e_j} m_{k-e_j,j} \oplus m_{k-e_j,j}, \end{aligned}$$

where $\oplus$ denotes bitwise exclusive-or (XOR).
The corresponding direction numbers are defined by

$$v_{k,j} = \frac{m_{k,j}}{2^k}.$$

Let $n \in \mathbb{N}_0$ with binary expansion

$$n = n_0 + 2n_1 + \cdots + 2^{r-1}n_{r-1}, \quad n_i \in \{0,1\}.$$

Then the j-th coordinate of the n-th Sobol' point is given by

$$x_{n,j} = n_0 v_{1,j} \oplus n_1 v_{2,j} \oplus \cdots \oplus n_{r-1} v_{r,j}.$$

The s-dimensional Sobol' point is

$$\boldsymbol{x}_n = (x_{n,1},\dots,x_{n,s}).$$

As stated in [14, Chapter 5], the Sobol' sequence is a low-discrepancy sequence.

Example 2.2.10 (First Elements of the Sobol' Sequence in 2D).
We fix $s = 2$ with primitive polynomials ordered by degree

$$p_1(x) = x + 1 \ (e_1 = 1,\ a_{1,1} = 1), \qquad p_2(x) = x^2 + x + 1 \ (e_2 = 2,\ a_{1,2} = a_{2,2} = 1),$$

and choose admissible initials

$$m_{1,1} = 1, \qquad m_{1,2} = 1, \quad m_{2,2} = 1.$$

Thus the first direction numbers are

$$v_{1,1} = \tfrac{1}{2}, \qquad v_{1,2} = \tfrac{1}{2}, \qquad v_{2,1} = \tfrac{3}{4}, \qquad v_{2,2} = \tfrac{1}{4}.$$

With $n = n_0 + 2n_1 + \cdots$ and $x_{n,j} = n_0 v_{1,j} \oplus n_1 v_{2,j} \oplus \cdots$, the first three points are

$$n = 0 : (n_0, n_1) = (0,0) \Rightarrow \boldsymbol{x}_0 = (0,0).$$
$$n = 1 : (1,0) \Rightarrow \boldsymbol{x}_1 = \left(\tfrac{1}{2}, \tfrac{1}{2}\right).$$
$$n = 2 : (0,1) \Rightarrow \boldsymbol{x}_2 = \left(\tfrac{3}{4}, \tfrac{1}{4}\right).$$

Remark 2.2.11.
Different valid choices of p_j and $m_{k,j}$ produce different – but equivalent – Sobol' sequences.

The bitwise structure of the Sobol' sequence makes it particularly efficient to generate and it enables streaming or online sampling in high dimensions.

2.2.3 Dimensional Performance and Sequence Comparison

The practical performance of low-discrepancy sequences is strongly influenced by the dimension s of the integration domain. Although both Halton and Sobol' sequences achieve the asymptotic star discrepancy bound of order $\mathcal{O}((\log N)^s / N)$, their behavior in higher dimensions differs significantly in practice due to the hidden constants.

Halton Sequence. While the Halton sequence performs well in low-dimensional settings, its structure leads to correlation artifacts in higher dimensions due to the use of increasing prime bases. These artifacts manifest as clustering or gaps in certain coordinate directions, which can degrade uniformity. As a result, the empirical discrepancy of the Halton sequence often grows faster than the theoretical bound suggests in moderate to high dimensions, as the hidden constant C depends on the dimension s[14, Section 2.4].

Sobol' Sequence. In contrast, the Sobol' sequence is explicitly constructed for high-dimensional performance. The use of carefully selected primitive polynomials and direction numbers minimizes inter-dimensional correlations. Furthermore, the bitwise construction allows for better numerical stability and efficient implementation. Sobol' sequences generally exhibit lower discrepancy than Halton sequences in dimensions $s > 10$, making them a standard choice in modern QMC applications.

The below Figure 2.3 illustrates the difference in a two-dimensional setting between the two low discrepancy sequences. The Sobol' sequence fills the unit square more uniformly, while the Halton sequence shows visible clustering.

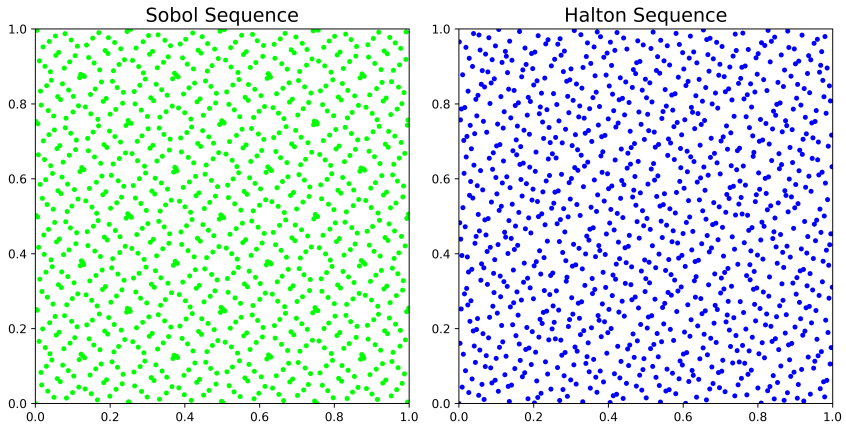


FIGURE 2.3: Comparison of Halton and Sobol' sequences in $s = 2$ dimensions for $N = 1024$ points. The Sobol' sequence fills the unit square more uniformly, demonstrating lower star discrepancy than the Halton sequence.

Remark 2.2.12.
In the experiments of Part II and Part III, the Sobol' and Halton sequences are not implemented manually. Instead, the implementation available in the Python package *SciPy* (version 1.16.0) through the module `scipy.stats.qmc` [31, 26] were used. The Sobol' and Halton sequences from this package though follow Definition 2.2.9 and Definition 2.2.6.

Chapter 3

Function Variation and Error Bounds in QMC

3.1 Function Variation

In Quasi-Monte Carlo (QMC) theory, the variation of a function plays a key role in bounding the integration error. Intuitively, variation quantifies how much a function oscillates or changes over its domain. This can be observed particularly well in the univariate total variation:

Definition 3.1.1 (Total Variation)**.**
Let $f : [a,b] \to \mathbb{R}$ be a real-valued univariate function. The *total variation* of f on the interval $[a,b] \subset \mathbb{R}$ is defined as

$$V_a^b(f) := \sup_{\mathcal{P}} \sum_{i=1}^{n} \left| f(x_i) - f(x_{i-1}) \right|,$$

where the supremum is taken over all partitions

$$\mathcal{P} = \{a = x_0 < x_1 < \cdots < x_n = b\},$$

of $[a,b]$.

While the total variation is sufficient for univariate functions, higher dimensions necessitate more refined notions. In this section, we follow the definitions of [20] and introduce two such notions: the Vitali variation and the variation in the sense of Hardy–Krause. These concepts are crucial for understanding the convergence guarantees provided by the Koksma–Hlawka inequality.

3.1.1 Variation in the Sense of Vitali

To express multidimensional variation concisely, we begin with the following notational conventions.

Definition 3.1.2 (Mixed Component Selection)**.**
Let $\boldsymbol{a},\boldsymbol{b} \in \mathbb{R}^s$ and let $u \subseteq \{1,\dots,s\}$ be an index set. We define

$$\boldsymbol{c} := \boldsymbol{a}^u : \boldsymbol{b}^{-u} \quad \text{with} \quad c_j := \begin{cases} a_j, & \text{if } j \in u, \\ b_j, & \text{if } j \notin u. \end{cases}$$

Here, the term $-u$ denotes the complement of u in $\{1,\dots,s\}$. Using this notation, the *alternating sum* is defined as follows:

Definition 3.1.3 (Alternating Sum)**.**
The *alternating sum* on s dimensions with respect to a function $f : [\boldsymbol{a},\boldsymbol{b}] \to \mathbb{R}$ on the hyperrectangle $[\boldsymbol{a},\boldsymbol{b}]$ is defined as

$$\Delta(f;\boldsymbol{a},\boldsymbol{b}) = \sum_{v \subseteq \{1,\dots,s\}} (-1)^{|v|} f(\boldsymbol{a}^v : \boldsymbol{b}^{-v}).$$

A classical approach to quantify function variation on hyperrectangles is based on the concept of *ladders*:

Definition 3.1.4 (Ladder)**.**
A *ladder* on the hyperrectangle $[\boldsymbol{a},\boldsymbol{b}]$ is of the form $\mathcal{Y} = \bigtimes_{j=1}^{s} \mathcal{Y}^j$, where $\times$ denotes the Cartesian product of sets. Each $\mathcal{Y}^j$ is a finite subset of $[a_j,b_j]$ for $j = 1,\dots,s$ where $\mathcal{Y}^j = \{y_1^j,\dots,y_k^j\}$ is ordered such that $a_j = y_1^j < y_2^j < \cdots < y_k^j$. The ladder could consist of a different number of points in each dimension.
The *successor* of a point $\boldsymbol{y} \in \mathcal{Y}$ in the ladder is defined componentwise as

$$y_+^j = \min\Big\{(y^j,\infty) \cap (\mathcal{Y}^j \cup \{b_j\})\Big\}.$$

with $\boldsymbol{y}_+ = (y_+^1,\dots,y_+^s)$.

$\mathbb{Y}$ denotes the set of all such ladders on $[\boldsymbol{a},\boldsymbol{b}]$.

Example 3.1.5. Consider the two-dimensional hyperrectangle $[\boldsymbol{a},\boldsymbol{b}] = [0,1]^2$ and the ladder $\mathcal{Y} = \{0,0.5\} \times \{0,0.25\}$. The ladder points are

$$\mathcal{Y} = \{(0,0),(0,0.25),(0.5,0),(0.5,0.25)\}.$$

The successors of these points are

$$\begin{aligned} (\ \ 0,\ \ \ 0)_+ &= (0.5,0.25), \\ (\ \ 0,0.25)_+ &= (0.5,1\ \ \ ), \\ (0.5,\ \ \ 0)_+ &= (1\ \ \ ,0.25), \\ (0.5,0.25)_+ &= (1\ \ \ ,1\ \ \ ). \end{aligned}$$

The visualization can be found in Figure 3.1.

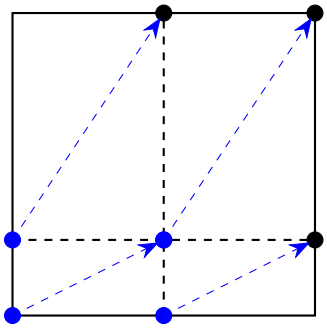


FIGURE 3.1: Visualization of the ladder $\mathcal{Y} = \{0, 0.5\} \times \{0, 0.25\}$ on the unit square $[0,1]^2$ and its successors.

Remark 3.1.6.
For a ladder $\mathcal{Y}$ on the hypercube $[\boldsymbol{a}, \boldsymbol{b}]$ the following equation holds:

$$\bigcup_{\boldsymbol{y} \in \mathcal{Y}} [\boldsymbol{y}, \boldsymbol{y}_+] = [\boldsymbol{a}, \boldsymbol{b}]$$

This is due to the ladder splitting the hypercube $[\boldsymbol{a}, \boldsymbol{b}]$ into axis-aligned rectangular subregions by the points $[\boldsymbol{y}, \boldsymbol{y}_+]$ for each $\boldsymbol{y} \in \mathcal{Y}$. Thus, the union of all such subregions is a disjoint cover of the hypercube.

The *variation with respect to a ladder* is then defined as:

Definition 3.1.7 (Variation with respect to a ladder)**.**
Let $\mathcal{Y} = \prod_{j=1}^{s} \mathcal{Y}^j$ be a ladder on $[\boldsymbol{a}, \boldsymbol{b}]$. Then the *variation* of a function f with respect to $\mathcal{Y}$ is

$$V_{\mathcal{Y}}(f) = \sum_{\boldsymbol{y} \in \mathcal{Y}} \left| \Delta(f; \boldsymbol{y}, \boldsymbol{y}_+) \right|.$$

The *Vitali variation* of a function is then the supremum of these ladder-based variations:

Definition 3.1.8 (Vitali Variation)**.**
The *Vitali variation* of a function f on the hyperrectangle $[\boldsymbol{a}, \boldsymbol{b}]$ is defined as

$$V_{[\boldsymbol{a}, \boldsymbol{b}]}(f) = \sup_{\mathcal{Y} \in \mathbb{Y}} V_{\mathcal{Y}}(f).$$

This variation captures the total fluctuation of f over axis-aligned rectangular subregions. It generalizes finite differences and serves as a foundation for defining the Hardy–Krause variation.

3.1.2 Hardy–Krause Variation

The *Hardy–Krause variation* refines the Vitali variation by summing variations over all lower-dimensional axis-aligned faces of the unit hypercube. It is defined as:

Definition 3.1.9 (Hardy–Krause Variation)**.**
The *Hardy–Krause variation* of a function f on the hyperrectangle $[\boldsymbol{a},\boldsymbol{b}]$ is defined as

$$V_{\mathrm{HK}}(f) = \sum_{u\subseteq\{1,\dots,s\}u\neq\emptyset} V_{[\boldsymbol{a}^u,\boldsymbol{b}^u]}\Big(f(x^u;\boldsymbol{b}^{-u})\Big). \tag{3.1}$$

Note that the Hardy–Krause variation reduces to the total variation in the univariate case. Importantly, $V_{\mathrm{HK}}(f) < \infty$ implies that f has bounded variation in all axis-aligned directions and projections.

Example 3.1.10.
Let $f(x) = \prod_{i=1}^{s} x_i$, the s-dimensional monomial on the hypercube $[0,1]^s$. Then it is clear that $f \in C^s$. According to Equation (6.2) and Remark 6.1.3 the calculation follows as:

$$\begin{aligned} V_{\mathrm{HK}}(f) &= \widehat{V}_{\mathrm{HK}}(f) \\ &= \sum_{\substack{u\subseteq\{1,\dots,s\}\\ u\neq\emptyset}} \int_{[0,1]^{|u|}} \left|\frac{\partial^{|u|}}{\partial x^u} f(x^u;1^{-u})\right| dx^u \\ &= \sum_{\substack{u\subseteq\{1,\dots,s\}\\ u\neq\emptyset}} \int_{[0,1]^{|u|}} 1\, dx^u \\ &= \sum_{\substack{u\subseteq\{1,\dots,s\}\\ u\neq\emptyset}} 1 \\ &= 2^s - 1\,. \end{aligned}$$

The notations will also be introduced in Chapter 6.

Example 3.1.11.
Consider $f_{d,r}(x) = \left(\max\left\{\sum_{i=1}^{d} x_i - \frac{1}{2}, 0\right\}\right)^r$. For $d > r+2$, this function has infinite Vitali and hence infinite Hardy–Krause variation. Such examples underline the limitations of QMC for functions with low smoothness or sharp nonlinearity [20, Prop. 16].

In the next section, the Koksma–Hlawka inequality will be presented, which highly depends on the concept of function variation introduced in this section. These examples illustrate the importance of regularity assumptions. In practice, many physically motivated maps – such as solutions to elliptic or parabolic PDEs – do exhibit sufficient smoothness to ensure bounded Hardy–Krause variation, justifying the use of QMC error bounds.

3.2 The Koksma–Hlawka Inequality

The Koksma–Hlawka inequality is a central result in Quasi-Monte Carlo theory. It provides a deterministic error bound for numerical integration that combines two distinct concepts: the discrepancy of the point set and the variation of the integrand. This inequality formalizes the intuition that more uniformly distributed sample points lead to smaller integration errors—provided the integrand is regular enough.

The inequality relates the QMC integration error to the star discrepancy of the point set and the Hardy–Krause variation of the integrand. A proof can be found in [3].

Theorem 3.2.1 (Koksma–Hlawka Inequality)**.**
Let $f : [0,1]^s \to \mathbb{R}$ be a function of bounded variation in the sense of Hardy–Krause and let $P = \{x_1, \dots, x_N\} \subset [0,1]^s$ be a finite point set. Then the integration error satisfies

$$\left| \frac{1}{N} \sum_{n=1}^{N} f(x_n) - \int_{[0,1]^s} f(x)\, dx \right| \leq D_N^*(P) \cdot V_{\mathrm{HK}}(f), \tag{3.2}$$

where $D_N^*(P)$ denotes the star discrepancy of the point set P and $V_{\mathrm{HK}}(f)$ is the Hardy–Krause variation of f.

Remark 3.2.2.
This bound is deterministic and non-asymptotic. It applies to any dimension s and makes no probabilistic assumptions on the sampling procedure. However, it requires f to have bounded variation in the Hardy–Krause sense, which excludes some classes of highly irregular or discontinuous functions.

The two factors on the right-hand side of the inequality – discrepancy and variation – capture complementary aspects of integration error.

- The **star discrepancy** $D_N^*(P)$ measures how uniformly the point set P samples the unit cube. It is determined entirely by the geometry of the sample points and vanishes if the empirical distribution matches the uniform distribution.
- The **Hardy–Krause variation** $V_{\mathrm{HK}}(f)$ reflects the total directional variation of the integrand f along all coordinate axes and lower-dimensional projections. It penalizes irregularity and ensures that the integrand behaves well under uniform sampling.

Both quantities are required: even a low-discrepancy point set cannot guarantee small error if the function has unbounded variation. Conversely, even a smooth function may incur a large error if the points cluster poorly.

Remark 3.2.3.
The Koksma–Hlawka inequality can be seen as a product-type estimate: each component controls a different source of error and their product bounds the total integration error.

The Koksma–Hlawka inequality provides valuable insight into the behavior of Quasi-Monte Carlo estimators:

- It motivates the use of low-discrepancy sequences such as Halton or Sobol', which minimize $D_N^*(P)$.
- It emphasizes the importance of function regularity—only functions with bounded Hardy–Krause variation are eligible for the bound.
- It establishes a worst-case, deterministic error guarantee, unlike the probabilistic bounds of Monte Carlo integration.

In practical terms, the Koksma-Hlawka inequality implies that for sufficiently smooth integrands with bounded Hardy–Krause variation and point sets obtained by low-discrepancy sequences, with star discprepancy of order (2.2) by definition, one can achieve integration errors fulfilling

$$\left| \frac{1}{N} \sum_{n=1}^{N} f(x_n) - \int_{[0,1]^s} f(x)\, dx \right| \leq \mathcal{O}\left(\frac{(\log N)^s}{N} \right),$$

which significantly improves over the $\mathcal{O}(N^{-1/2})$ rate of standard Monte Carlo methods. This improvement, however, comes at the cost of stronger assumptions and increased sensitivity to dimensionality.

Building upon this theoretical foundation, the next parts of this thesis turn from abstract principles to concrete applications. Part II investigates how QMC methods can enhance the training of neural networks, while Part III applies these techniques to the simulation of X-ray images in medical imaging.

Part II

Quasi-Monte Carlo Sampling for Neural Network Training

Chapter 4

Motivation and Problem Formulation

The second part of this thesis investigates the application of Quasi-Monte Carlo (QMC) sampling techniques to enhance the training of Deep Neural Networks (DNNs) in *many-query problems* – computational tasks in which a parameterized map must be evaluated for a large number of inputs, each requiring costly simulations or numerical solutions of PDEs. This situation arises frequently in scientific computing, e.g. in uncertainty quantification, Bayesian inversion, optimal control and model calibration. This part of the thesis thus addresses a current topic, specifically the article by Mishra and Rusch [18].

Such problems motivate the use of surrogate models, typically based on deep neural networks, which are trained offline on a set of inputs and then used for fast evaluation online. However, using Monte Carlo (MC) sampling for generating training data may yield to slow convergence as introduced in Part I.

To overcome this limitation, this thesis explores the use of low-discrepancy sequences to generate training data which fill the input domain more uniformly. Under mild regularity assumptions, they can significantly reduce the generalization error due to improved equidistribution properties.

4.1 Many-Query Problems in Scientific Computing

Many-query problems refer to scenarios in which a computationally expensive model $f(x)$ must be evaluated for many different parameter values $x \in \mathcal{X} \subset \mathbb{R}^s$. Typical applications include:

- Uncertainty quantification (UQ)
- Optimal design/control
- Inverse problems

The sheer cost of evaluating $f(x)$ (e.g., via numerical PDE solvers) motivates the use of surrogate models. Once trained, a surrogate $\hat{f}$ can provide rapid predictions for unseen inputs at a fraction of the computational cost.

4.2 The Role of Function Approximation

We formalize the problem as learning a map $f\colon [0,1]^s \to \mathbb{R}^m$ from data. The assumption of a unit hypercube domain reflects common practice in QMC theory and is justified via homeomorphic mappings from more general compact and simply connected input spaces.

In this thesis, the surrogate model $\hat{f}$ will be chosen from the class of deep neural networks and is trained on data $\{(x_i, f(x_i))\}_{i=1}^N$. The aim is to approximate f with high accuracy while minimizing the number of required evaluations.

Deep Neural Networks are known to be universal approximators. Under mild conditions on the later introduced *activation functions* they proved to be able to approximate any continuous function on compact sets arbitrarily well [6].

4.3 Applicability of the Koksma-Hlawka Inequality to DNN Training

Generally, training data is generated using Monte Carlo sampling, where points x_i are drawn independently and identically distributed (i.i.d.) according to a probability measure μ on $[0,1]^s$. Under this approach, statistical learning theory provides an estimate for the expected generalization error $\mathbb{E}[\mathcal{E}_G]$ as follows:

$$\mathcal{E}_G \sim \mathcal{E}_T + \mathcal{O}\left(\frac{1}{\sqrt{N}}\right),$$

where $\mathcal{E}_T$ is the empirical training error. Both, the generalization and training error will be formally introduced in Section 5.2. However, as argued in [18], this rate is unsatisfactory in many-query problems because:

- A small generalization error requires a large N, which is costly due to expensive evaluations of $f(x)$.
- The constant in the bound depends on the variance and correlations in the training process, which are hard to control.

A promising alternative to random sampling is to use Quasi-Monte Carlo (QMC) sampling. Low-discrepancy sequences, as the introduced Sobol' or Halton sequence, provide a way to sample the domain $[0,1]^s$ more uniformly than random sampling. In Quasi-Monte Carlo theory, the Koksma-Hlawka inequality (Theorem 3.2.1) applied to the difference $f - \hat{f}$ gives an error bound of the form

$$|\mathcal{E}_G - \mathcal{E}_T| \leq V_{\mathrm{HK}}(|f - \hat{f}|) \cdot D_N^* \leq V_{\mathrm{HK}}(|f - \hat{f}|) \cdot C_s \cdot \frac{(\log N)^s}{N}.$$

The left part of the inequality will be formally derived in Theorem 6.2.1. The right part follows from the known discrepancy for low-discrepancy sequences. It is not immediately clear, whether the Koksma–Hlawka inequality can be applied to the generalization gap $|\mathcal{E}_G - \mathcal{E}_T|$ in the context of training DNNs. This is because the Koksma-Hlawka inequality requires the integrand to have bounded variation in the sense of Hardy–Krause to ensure a finite bound.

In the following chapters, it will be shown that under mild assumptions on the surrogate model $\hat{f}$ and that the ground truth function f has bounded variation in the

sense of Hardy–Krause, the Koksma–Hlawka inequality is applicable to the generalization gap. In particular, it will be demonstrated that using low-discrepancy sequences for generating training data can significantly reduce the generalization gap.

Since the direct application of the Koksma–Hlawka inequality, in particular the determination of the Hardy–Krause variation, is not practical for general formulations of the approximation function $\hat{f}$ depending on the training data, the usefulness and applicability of the approach will be verified in Chapter 7 through selected examples.

Chapter 5

Theoretical Foundations of Deep Neural Networks

Deep Neural Networks (DNNs) have become a powerful class of surrogate models in scientific computing, particularly for high-dimensional and expensive-to-evaluate functions. This chapter introduces the mathematical structure and training procedure of DNNs, with a focus on activation functions, generalization theory and gradient-based optimization algorithms. It follows the framework established in [18].

5.1 Theoretical Foundations of Deep Neural Networks

The key feature enabling neural networks to approximate nonlinear functions is the application of scalar activation functions. We begin by introducing these components before defining the class of Deep Neural Network functions.

Definition 5.1.1 (Activation Function)**.**
An *activation function* is a measurable function $\sigma : \mathbb{R} \to \mathbb{R}$, typically nonlinear, that is applied componentwise to the output of each layer in a neural network.

Remark 5.1.2.
In general the activation function σ is 1-dimensional, i.e. $\sigma : \mathbb{R} \to \mathbb{R}$. However, it is common to apply it componentwise to vectors and extend the definition to $\sigma : \mathbb{R}^k \to \mathbb{R}^k$ with $\sigma(z) = (\sigma(z_1), \ldots, \sigma(z_k))^\top$.

Example 5.1.3 (Common Activation Functions).
Two widely used activation functions are given below and illustrated in Figure 5.1. These are also the activation functions we employ in the experimental evaluations presented in Chapter 7.

- **Tanh**:
$$\sigma(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$
The hyperbolic tangent function maps real numbers to $(-1, 1)$ and is differentiable everywhere. It is often used in hidden layers of neural networks.

- **Sigmoid**:
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid function maps real numbers to $(0,1)$ and is differentiable everywhere. It is often used in theoretical analysis and binary classification tasks.

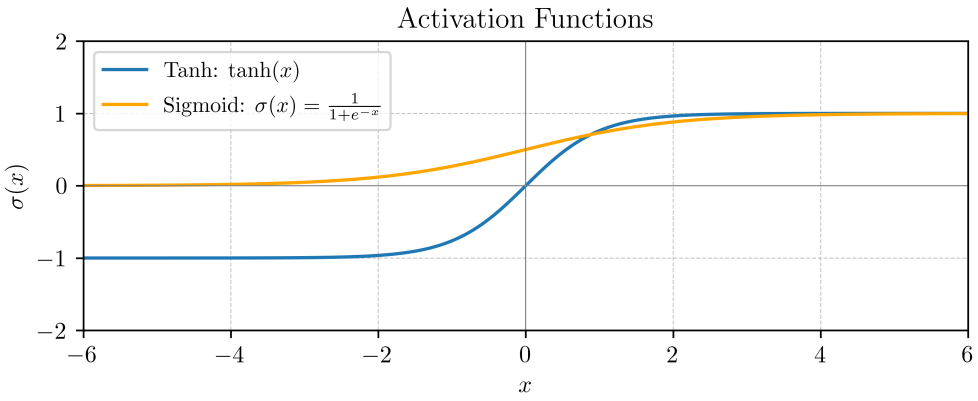


FIGURE 5.1: Comparison of ReLU and Sigmoid activation functions.

With an activation function in place, we now define the term Deep Neural Network by introducing the representing function that a fully connected feedforward neural network is expressing[18].

Definition 5.1.4 (Fully Connected Deep Neural Network)**.**
Let $L \in \mathbb{N}$ and $\mathbf{d} = (d_0, d_1, \ldots, d_L)$ be a sequence of positive integers, where $d_0 = s$ is the input dimension and $d_L = 1$ the output dimension. A Deep Neural Network (DNN) of *depth* L and *layer widths* $d_1, \ldots, d_{L-1}$ is a function $\hat{f}_\theta : \mathbb{R}^s \to \mathbb{R}$ of the form

$$\hat{f}_\theta(y) = A_L \circ \sigma \circ A_{L-1} \circ \cdots \circ \sigma \circ A_1(y), \tag{5.1}$$

where each *layer map* $A_l : \mathbb{R}^{d_{l-1}} \to \mathbb{R}^{d_l}$ is an affine transformation

$$A_l(z) = W_l z + b_l,$$

with weights $W_l \in \mathbb{R}^{d_l \times d_{l-1}}$ and biases $b_l \in \mathbb{R}^{d_l}$. The activation function $\sigma : \mathbb{R} \to \mathbb{R}$ is applied componentwise.
The collection of all trainable parameters is denoted by

$$\theta = \{(W_l, b_l)\}_{l=1}^{L},$$

and fully determines the function $\hat{f}_\theta$ for a fixed activation function σ. The set of all such realizable functions for a given architecture $\mathbf{d}$ and activation function σ is denoted by $\mathcal{F}_{\mathbf{d},\sigma}$.

The architectural parameters are interpreted as follows:

- The *depth* L is the number of affine transformations.
- The number of *hidden layers* is $L-1$.
- The *width* d_l specifies the number of neurons in layer l.
- The *layer structure* is encoded in the tuple $\mathbf{d}$.
- The *activation function* σ usually introduces nonlinearity.

As common in literature, the *number of layers* is referenced by the number of affine transformations L and coincides with the term *depth*. The input layer is not counted as a layer in this context. In Figure 5.2, a fully connected DNN with constant width is illustrated.

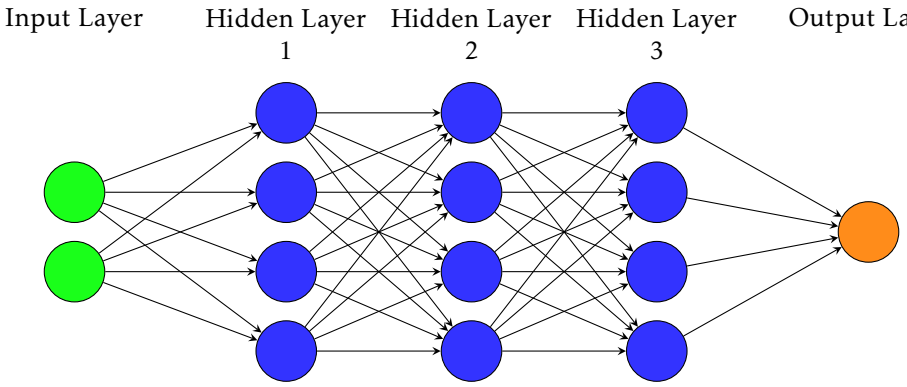


FIGURE 5.2: Schematic of a fully connected Deep Neural Network with constant width $m = 4$ and depth $L = 4$ layers. Each circular node represents an artificial neuron. The arrows represent the connections between neurons in adjacent layers.

Remark 5.1.5.
For common DNN architectures with L layers, a constant number of neurons m in each layer, input dimension $d_0 = s$ and output dimension $d_L = 1$ the layer structure follows:

$$d = (d_0, d_1, ..., d_{L-1}, d_L) = (s, m, \dots, m, 1)$$

Thus, for each affine transformation $l \in \{1, ..., L\}$, one counts $d_{l-1} \cdot d_l$ weights and d_l biases. Hence, the total number of trainable parameters (weights and biases) θ in the network is given by

$$\begin{aligned} N_\theta &= \sum_{l=1}^{L} (d_{l-1} \cdot d_l + d_l) \\ &= (s \cdot m + m) + \sum_{l=2}^{L-1} (m \cdot m + m) + (m \cdot 1 + 1) \\ &= (L-2) \cdot m^2 + (L+s) \cdot m + 1. \end{aligned}$$

5.2 DNN Evaluation

The training of a DNN involves supervised learning, where the goal is to minimize the difference between the surrogate model $\hat{f}$ and the ground truth function f. Supervised training of a DNN therefore requires minimizing the difference between the network output $\hat{f}_\theta(y)$ and a ground truth function $f(y)$ on a given training set $\{y_i\}_{i=1}^N$.

We start by defining the loss function which is used to measure this difference during the training:

Definition 5.2.1 (Loss Function)**.**
Let $f : [0,1]^s \to \mathbb{R}$ be a ground truth function defined on the hypercube and $\hat{f}_\theta$ a respective surrogate with network architecture $\mathbf{d}$, according parameters θ and activation function σ. Then, given the surrogate and a respective point set $\mathcal{P}$, the *loss function* is defined as

$$\mathcal{L}_\mathcal{P}(\theta) := \frac{1}{|\mathcal{P}|} \sum_{y \in \mathcal{P}} |f(y) - \hat{f}_\theta(y)|^p ,$$

for some $1 \leq p < \infty$.

The point set $\mathcal{P}$ in the loss function is usually given by the training set. By defining the loss on the training data, the *training error* $\mathcal{E}_T$ is defined:

Definition 5.2.2 (Training Error)**.**
Let $f : [0,1]^s \to \mathbb{R}$ be a ground truth function defined on the hypercube and $\hat{f}_\theta$ a respective surrogate model with architecture $\mathbf{d}$, according parameters θ and activation function σ. Then, given the matching training set $\{(y_i, f(y_i))\}_{i=1}^N$ with $y_i \in [0,1]^s$ of the surrogate model, the *training error* is defined as

$$\mathcal{E}_\mathrm{T}(\theta) := \frac{1}{N} \sum_{i=1}^{N} |f(y_i) - \hat{f}_\theta(y_i)|.$$

The *generalization error* $\mathcal{E}_\mathrm{G}$ is evaluating the difference between the surrogate model $\hat{f}_\theta$ and the ground truth function f on the entire domain $[0,1]^s$.

Definition 5.2.3 (Generalization Error)**.**
Let $f : [0,1]^s \to \mathbb{R}$ be a ground truth function defined on the hypercube and $\hat{f}_\theta$ a respective surrogate with nework architecture $\mathbf{d}$, according parameters θ and activation function σ. Then, given the surrogate, the *generalization error* is defined as:

$$\mathcal{E}_\mathrm{G}(\theta) := \int_{[0,1]^s} |f(y) - \hat{f}_\theta(y)| \, \mathrm{d}y.$$

Remark 5.2.4.
Whereas the training error $\mathcal{E}_\mathrm{T}$ is numerically straightforward to compute, the generalization error $\mathcal{E}_\mathrm{G}$ is often not directly computable, as the ground truth function f could not be known besides the training data or is costly to compute.

Definition 5.2.5 (Generalization Gap)**.**
The *generalization gap* for a given DNN architecture $\mathbf{d}$, according parameters θ and activation function σ is the difference between the generalization and training error:

$$|\mathcal{E}_\mathrm{G}(\theta) - \mathcal{E}_\mathrm{T}(\theta)|.$$

5.3 Optimization and Training Procedure

In practice, an initial surrogate DNN $\hat{f}_\theta$ is generated with initial parameters θ. In supervised learning, the goal is to minimize the *regularized loss function* with respect to the parameters θ by adapting the weights of the network during the training process. Regularization is introduced to counteract *overfitting*, i.e. the tendency of a highly flexible network to reproduce noise of the training data instead of the underlying function. The regularization term penalizes certain properties of the parameters, thereby enforcing additional structure on the solution. For instance, an L^2 penalty discourages large weight magnitudes and effectively promotes smoother functions.

Thus, one aims to find a solution to the following minimization problem:

$$\theta^* = \arg\min_\theta \mathcal{J}_\mathcal{P}(\theta), \quad \text{with } \mathcal{J}(\theta) = \mathcal{L}_\mathcal{P}(\theta) + \lambda \cdot R(\theta),$$

where $\mathcal{P}$ is the training set, $\mathcal{L}_\mathcal{P}$ is the empirical loss (e.g. mean squared error), $R(\theta)$ is a regularization functional and $\lambda > 0$ is the regularization parameter balancing fidelity to the training data and the strength of the imposed constraint.

The training is carried out on a given *training dataset* consisting of N input–output pairs $(x_n, f(x_n))_{n=1}^N$, where N is called the *training set size*. Since the dataset is usually too large to be processed at once, it is divided into smaller subsets of fixed cardinality B, called *mini-batches*. The number B is referred to as the *batch size* and determines how many samples are processed simultaneously in each training step.

The training procedure is organized into multiple passes over the dataset, called *epochs*. At the beginning of each epoch $e \in 1, \dots, E$, the dataset is typically shuffled and then partitioned into mini-batches

$$(\mathcal{B}_k^e)_{k=1}^K, \quad \text{with } K = \left\lceil \frac{N}{B} \right\rceil,$$

such that the union of all mini-batches corresponds to the full training set. This ensures that in each epoch, the entire dataset is used for training.

For each mini-batch $\mathcal{B}_k^e$, the algorithm executes the following steps: the current surrogate network $\hat{f}_\theta$ is applied to all samples in $\mathcal{B}_k^e$ in a forward pass and the corresponding *regularized loss function* $\mathcal{J}_{\mathcal{B}_k^e}$ is evaluated with respect to these samples. The gradient of the loss with respect to the parameters, $\nabla_\theta \mathcal{J}_{\mathcal{B}_k^e}$, is then computed by backpropagation. Finally, the parameters θ are updated according to the chosen optimization algorithm Opt, e.g. SGD, Adam or Lion as introduced in the followup subsections.

This procedure is repeated over all mini-batches of all epochs or until a stopping criterion is met, resulting in the final trained network $\hat{f}_{\theta^*} = \hat{f}_{\theta_E}$.

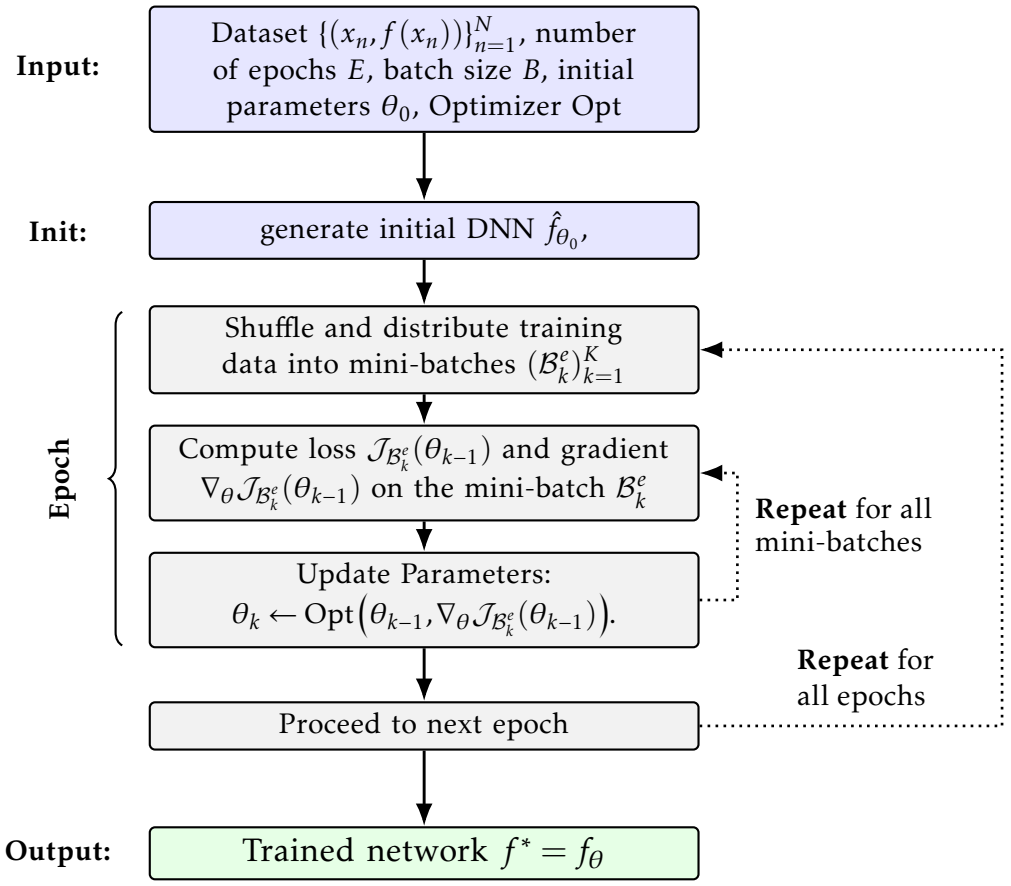


FIGURE 5.3: Generic DNN training: for each of E epochs, iterate over mini-batches of size B, compute loss/gradients and update parameters using optimizer Opt.

In the following two subsections, two commonly used optimization algorithms in DNN training are shortly introduced: Adaptive Moment Estimation (Adam) and Lion. As the main focus of this thesis is the comparison of different sampling strategies and not the optimization algorithms, we do not go into detail on the mathematical properties of these algorithms. For a more detailed introduction to optimization algorithms in DNNs, see [8, Chapter 8].

5.3.1 Stochastic Gradient Descent (SGD)

The very generic Stochastic Gradient Descent (SGD) algorithm is a first-order optimization method that updates the parameters θ of the DNN by computing

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta \mathcal{J}_{\mathcal{B}}(\theta_t),$$

where $\eta > 0$ is the fixed learning rate and $\mathcal{J}_{\mathcal{B}}$ is the generalized loss function evaluated on the mini-batch. There also exist varaiants of the SGD optimization algorithm, which adaptively change the learning rate η during the training process.

5.3.2 Adam Optimizer

The Adaptive Moment Estimation (Adam) algorithm [8] is a variant of stochastic gradient descent (SGD) that is designed to work well *out of the box*. It combines two ideas:

1. *momentum*, which smooths noisy gradients by averaging them over time,
2. *coordinatewise adaptive step sizes*, which scale the update of each parameter according to how large its recent gradients have been.

Mini-batch gradient. At iteration t we draw a mini-batch $\mathcal{B}_t$ from the training set and compute the mini-batch loss

$$\mathcal{J}_{\mathcal{B}_t}(\theta_{t-1}),$$

with gradient

$$g_t = \nabla_\theta \mathcal{J}_{\mathcal{B}_t}(\theta_{t-1}).$$

Running averages ("moments"). Adam maintains two exponential moving averages of the gradients: the first-order moment (mean) m_t and the second-order moment (uncentered variance) v_t:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{5.2}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{5.3}$$

where the square and all operations are elementwise and $\beta_1, \beta_2 \in [0,1)$ are decay parameters (typical defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$). Intuitively, m_t smooths the direction of descent (momentum), while v_t tracks how large recent gradients have been for each parameter.

Bias correction. Because $m_0 = v_0 = 0$, early averages are biased toward zero. Adam corrects this via

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \tag{5.4}$$
$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{5.5}$$

which are unbiased estimates of the first and second moments under the exponential averaging model.

Adaptive update. Parameters are then updated elementwise using

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

where η is a global learning rate and ϵ (e.g. 10^{-8}) ensures numerical stability. The numerator behaves like momentum; the denominator normalizes by a root-mean-square of recent gradients, giving each parameter its own effective step size.

Remark 5.3.1.
This combination makes Adam robust in high-dimensional, noisy settings and when gradients have very different scales across parameters. Performance and convergence depend on the hyperparameters $(\beta_1, \beta_2, \eta, \epsilon)$ and on standard training choices (batch size, schedule, etc.).

Algorithm 1 Adam Optimizer

Initialize θ_0, $m_0 \leftarrow 0$, $v_0 \leftarrow 0$; choose $\eta > 0$, $\beta_1, \beta_2 \in [0, 1)$, $\epsilon > 0$
for $t = 1$ to T **do**
  Sample mini-batch $\mathcal{B}_t$ and set $g_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1}; \mathcal{B}_t)$
  $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$
  $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$
  $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
  $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
  $\theta_t \leftarrow \theta_{t-1} - \eta\, \hat{m}_t / \left(\sqrt{\hat{v}_t} + \epsilon\right)$
end for

5.3.3 Lion Optimizer

The *Lion* optimizer (**EvoLved Sign Momentum**) [4] is a recently discovered first-order optimization algorithm for training deep neural networks. Unlike adaptive methods such as Adam, which maintain both first- and second-moment estimates of gradients, Lion keeps track only of momentum and applies a *sign-based* update. This leads to a very lightweight algorithm with strong empirical performance across large-scale vision, language and multimodal tasks.

Key ideas. Lion builds on two simple components:

1. *Momentum*, maintained as an exponential moving average of past gradients,
2. a *sign update*, which discards the magnitude and uses only the direction of the combined momentum and gradient.

The sign operation ensures that every parameter update has the same step size in magnitude, making the algorithm robust to rescaling of gradients. At the same time, the momentum term provides stability by smoothing out noisy updates.

Mini-batch gradient. As in other stochastic optimizers, at iteration t a mini-batch $\mathcal{B}_t$ is sampled and the stochastic gradient is computed:

$$g_t \;=\; \nabla_\theta \mathcal{J}_{\mathcal{B}_t}(\theta_{t-1}).$$

Momentum update. Lion maintains an exponential moving average of past gradients:

$$m_t \;=\; \beta_1 m_{t-1} + (1 - \beta_1)\, g_t,$$

where $\beta_1 \in [0, 1)$ controls how quickly past gradients decay. A common choice is $\beta_1 = 0.9$.

Parameter update. The parameter step is determined by the sign of a linear combination of current momentum and gradient:

$$\theta_t \;=\; \theta_{t-1} - \eta \cdot \operatorname{sign}\Big(\beta_2 m_t + (1-\beta_2) g_t\Big),$$

where η is the learning rate and β_2 balances between the momentum term and the current gradient. The `sign` function is applied elementwise so that each parameter receives a step of equal magnitude η, with the direction determined by the gradient information.

Remark 5.3.2.
Lion differs from Adam and similar adaptive methods by eliminating the second-moment accumulation v_t, reducing memory usage and computation. The use of the sign function makes updates invariant to gradient scale, while the combination of m_t and g_t ensures stability. Because the sign update yields larger effective step norms, Lion typically requires a smaller learning rate (often 3–10× smaller than AdamW) and a larger decoupled weight decay to achieve comparable regularization strength. Empirically, Lion has been shown to match or outperform AdamW on diverse large-scale benchmarks.

Algorithm 2 Lion Optimizer

Initialize θ_0, $m_0 \leftarrow 0$; choose $\eta > 0$, $\beta_1, \beta_2 \in [0, 1)$
for $t = 1$ to T **do**
    Sample mini-batch $\mathcal{B}_t$ and set $g_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1}; \mathcal{B}_t)$
    $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1)\, g_t$
    $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \operatorname{sign}\Big(\beta_2 m_t + (1-\beta_2) g_t\Big)$
end for

Chapter 6

QMC-Based Deep Learning Algorithm

The goal of this chapter is to mathematically analyze the function approximation problem and find bounds on the generalization gap of DNN surrogate models obtained by low-discrepancy training data. The analysis is based on the Hardy–Krause variation and the Koksma-Hlawka inequality introduced in Chapter 3.

6.1 Preliminaries

For the sake of completeness, we start with some results on approximations of variations defined in Section 3.1. First, we recall an important identity for the alternating sum.

The following results come from [20, Section 9]. For completeness, the conceptual proofs are given here in full.

Proposition 6.1.1.
Let f be a function defined on the hypercube $[a,b] \subset \mathbb{R}^s$ with existing and integrable derivative $\frac{\partial^s}{\partial x_1 \cdots \partial x_s} f$. Then, the following holds for the alternating sum:

$$\int_{[a,b]} \frac{\partial^s}{\partial x_1 \cdots \partial x_s} f(x)\, dx = \Delta(f;a,b) \tag{6.1}$$

where $\Delta(f;a,b)$ is the alternating sum from Definition 3.1.3.

Proof.
The proof follows by induction on the dimension s.

Base case ($s = 1$).

$$\int_a^b \frac{\partial f(x)}{\partial x}\, dx = f(b) - f(a) = \Delta(f;a,b).$$

Thus, the statement holds for $s = 1$.

Induction hypothesis.
Assume the statement holds for dimension $s-1$.

Induction step ($s-1 \to s$)**.**
Write $x = (x', x_s)$ with $x' \in [a', b'] := \prod_{j=1}^{s-1} [a_j, b_j]$. First, apply the one-dimensional fundamental theorem of calculus in the last coordinate:

$$\int_{a_s}^{b_s} \frac{\partial^s f(x', t)}{\partial x_1 \cdots \partial x_s} \, dt = \frac{\partial^{s-1} f(x', b_s)}{\partial x_1 \cdots \partial x_{s-1}} - \frac{\partial^{s-1} f(x', a_s)}{\partial x_1 \cdots \partial x_{s-1}}.$$

Now integrate over x' and use Fubini's theorem:

$$\int_{[a,b]} \frac{\partial^s f(x)}{\partial x_1 \cdots \partial x_s} \, dx = \int_{[a',b']} \frac{\partial^{s-1} f(x', b_s)}{\partial x_1 \cdots \partial x_{s-1}} \, dx' - \int_{[a',b']} \frac{\partial^{s-1} f(x', a_s)}{\partial x_1 \cdots \partial x_{s-1}} \, dx'.$$

We now apply the induction hypothesis to both integrals (for the functions $g_1(x') := f(x', b_s)$ and $g_2(x') := f(x', a_s)$):

$$\int_{[a',b']} \frac{\partial^{s-1} f(x', b_s)}{\partial x_1 \cdots \partial x_{s-1}} \, dx' = \Delta\Big(f(\cdot, b_s); a', b'\Big),$$

$$\int_{[a',b']} \frac{\partial^{s-1} f(x', a_s)}{\partial x_1 \cdots \partial x_{s-1}} \, dx' = \Delta\Big(f(\cdot, a_s); a', b'\Big).$$

Thus,

$$\int_{[a,b]} \frac{\partial^s f(x)}{\partial x_1 \cdots \partial x_s} \, dx = \Delta\Big(f(\cdot, b_s); a', b'\Big) - \Delta\Big(f(\cdot, a_s); a', b'\Big).$$

This is exactly the recursive definition of the corner difference in s dimensions. Hence, the statement holds for s. □

This yields another result for the variation in the sense of Vitali on functions with existing and integrable derivative.

Corollary 6.1.2.
Let f be a function defined on the hypercube $[a, b] \subset \mathbb{R}^s$ with existing and integrable derivative $\frac{\partial^s}{\partial x_1 \cdots \partial x_s} f$. Then, the following holds for the variation in the sense of Vitali:

$$V_{[a,b]}(f) \leq \int_{[a,b]} \left| \frac{\partial^s}{\partial x_1 \cdots \partial x_s} f(x) \right| dx.$$

Proof.
Recalling the definition of variation with respect to a ladder from Definition 3.1.7 and applying Proposition 6.1.1 yields

$$V_{\mathcal{Y}}(f) = \sum_{y \in \mathcal{Y}} \left| \Delta(f; y, y_+) \right| = \sum_{y \in \mathcal{Y}} \left| \int_{[y, y_+]} \frac{\partial^s}{\partial x_1 \cdots \partial x_s} f(x) \, dx \right|.$$

Applying the triangle inequality then leads to

$$\sum_{y\in\mathcal{Y}}\left|\int_{[y,y_+]}\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\,dx\right|\leq\sum_{y\in\mathcal{Y}}\int_{[y,y_+]}\left|\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\right|dx$$

Since for a ladder $\mathcal{Y}$ of a hyperrectangle $[a,b]$ the equality $\bigcup_{y\in\mathcal{Y}}[y,y_+]=[a,b]$ holds per definition, the sum builds the integral over the whole hyperrectangle $[a,b]$:

$$\sum_{y\in\mathcal{Y}}\int_{[y,y_+]}\left|\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\right|dx=\int_{[a,b]}\left|\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\right|dx.$$

Therefore

$$V_{[a,b]}(f)=\sup_{\mathcal{Y}\in\mathbb{Y}}V_{\mathcal{Y}}(f)\leq\int_{[a,b]}\left|\frac{\partial^s}{\partial x_1\cdots\partial x_s}f(x)\right|dx.$$

□

Remark 6.1.3.
As computing the Hardy–Krause variation is not tractable, the following upper bound follows immediately from the preceding results by Corollary 6.1.2 and Definition 3.1.9 of the Hardy–Krause variation:

$$V_{\mathrm{HK}}(f)\leq\widehat{V}_{\mathrm{HK}}(f):=\sum_{\substack{u\subseteq\{1,\dots,s\}\\u\neq\emptyset}}\int_{[a^u,b^u]}\left|\frac{\partial^{|u|}}{\prod_{j\in u}\partial x_j}f(x^u;b^{-u})\right|dx^u\,. \tag{6.2}$$

Assuming $\widehat{V}_{\mathrm{HK}}(f)<\infty$ implies that all derivatives in the sum $\frac{\partial^{|u|}}{\partial x^u}f(x^u;b^{-u})$ for all $\emptyset\neq u\subseteq\{1,\dots,s\}$ exist.

If all derivatives in the sum exist and are continuous, then by [20, Section 9] it follows that:

$$V_{\mathrm{HK}}(f)=\widehat{V}_{\mathrm{HK}}(f)\,.$$

6.2 Error estimation of DNN surrogates obtained by low-discrepancy training data

Given the computable training error $\mathcal{E}_T$ depending on the training data, we now seek to estimate the generalization error $\mathcal{E}_G$. To get there, we will use an estimate of the generalization gap and follow [18].

Theorem 6.2.1.
Assume that $\hat{f}$ is a surrogate model obtained by DNN training of a mapping f on the hypercube $[0,1]^s$ and $\mathcal{S} = \{x_n\}_{n=1}^N \subset [0,1]^s$ is a low-discrepancy sequnece. Further, let $V_{\mathrm{HK}}(|f-\hat{f}|) < \infty$. Then, the following estimate on the generalization gap holds:

$$|\mathcal{E}_G - \mathcal{E}_T| \leq C_s \cdot \frac{V_{\mathrm{HK}}(|f-\hat{f}|)(\log N)^s}{N},$$

where C_s is a constant and $V_{\mathrm{HK}}(|f-\hat{f}|)$ is the Hardy–Krause variation from Definition 3.1.9.

Proof.
This simply follows from the Koksma-Hlawka inequality (Theorem 3.2.1) applied to the difference $f - \hat{f}$:

$$\begin{aligned}
|\mathcal{E}_G - \mathcal{E}_T| &= \left| \int_{[0,1]^s} f(x) - \hat{f}(x)\, dx - \frac{1}{N} \sum_{n=1}^{N} \left[f(x_n) - \hat{f}(x_n) \right] \right| \\
&\leq V_{\mathrm{HK}}(|f-\hat{f}|) \cdot D_N^* \\
&\leq C_s \cdot \frac{V_{\mathrm{HK}}(|f-\hat{f}|)(\log N)^s}{N}
\end{aligned}$$

Here, D_N^* is the star-discrepancy of the low-discrepancy sequence $\mathcal{S}$ and the constant C_s, associated with the discrepancy of the sequence $\mathcal{S}$, is dependent on the dimension s. □

Theorem 6.2.1 states that the generalization gap can be bounded by the Hardy–Krause variation of the difference $|f-\hat{f}|$ and the discrepancy of the training data. However, it is still not clear under what conditions the Hardy–Krause variation of the difference $|f-\hat{f}|$ is bounded. The following theorem states sufficient conditions for the boundedness of the Hardy–Krause variation of the difference $|f-\hat{f}|$.

Theorem 6.2.2.
Let $f : [0,1]^s \to \mathbb{R}$ be a map with $\widehat{V}_{HK}(f) < \infty$ and $\frac{\partial^{|v|}}{\prod_{j\in v} \partial x_j} f(x^v; 1^{-v})$ for all $\emptyset \neq v \subseteq \{1,\ldots,s\}$ is continuous. Further, let $\hat{f}$ be the DNN surrogate with an activation function $\sigma \in C^s$ trained on a low-discrepancy sequence $\mathcal{S} = \{x_n\}_{n=1}^N \subset [0,1]^s$. Then for a given tolerance $\delta > 0$ the following holds:

$$\mathcal{E}_G \leq \mathcal{E}_T + C \cdot \frac{(\log N)^s}{N} + 2\delta.$$

The constant C depends on the training algorithm and the tolerance δ associated with the mollifier introduced in the proof.

To proof this theorem, we will use the following proposition stating a multivariate version of the Faà di Bruno formula from [10].

Proposition 6.2.3 (Multivariate Faà di Bruno formula)**.**
Let $h : \mathbb{R}^s \to \mathbb{R}^{d_1}$ with continuous derivative $\frac{\partial^s}{\partial x_1 \cdots \partial x_s} h$ and $g \in C^s(\mathbb{R}^{d_1}, \mathbb{R}^{d_2})$ be two functions. Then the following derivative of the composition $g \circ h$ follows:

$$\frac{\partial^s}{\partial x_1 \cdots \partial x_s}(g \circ h)(x) = \sum_{\pi} \left(g^{(|\pi|)} \circ h\right)(x) \cdot \prod_{B \in \pi} \frac{\partial^{|B|}}{\prod_{j \in B} \partial x_j} h(x)$$

Here, the sum is taken over all partitions π of the set $\{1, \ldots, s\}$ and B are the sets within the partition π.

A proof of the Multivariate Faà di Bruno formula can be found in [10, Proposition 1].

The multivariate Faà di Bruno formula will be used to proof the following result on the conditions for boundedness of the Hardy–Krause variation of compositions of functions.

Corollary 6.2.4.
Let $h : [a, b] \to \mathcal{X}$ for a hyperrectangle $[a, b] \in \mathbb{R}^s$ and $\mathcal{X} \subset \mathbb{R}$. Further, assume that $\widehat{V}_{\mathrm{HK}}(h) < \infty$ and $\frac{\partial^{|v|}}{\prod_{j \in v} \partial x_j} h(x^v; b^{-v})$ for all $\varnothing \neq v \subseteq \{1, \ldots, s\}$ is continuous. Then, the Hardy–Krause variation of the composition $g \circ h$ is bounded

$$V_{\mathrm{HK}}(g \circ h) < \infty$$

for all $g \in C^s(\mathbb{R})$.

Proof.
To check the boundedness of the Hardy–Krause variation of a composition of two functions, first Corollary 6.1.2 is applied on the Hardy–Krause variation of the composition $g \circ h$:

$$\begin{aligned} V_{\mathrm{HK}}(g \circ h) &= \sum_{\substack{v \subseteq \{1, \ldots, s\} \\ v \neq \varnothing}} V_{[a^v, b^v]}(g \circ h)(x^v; b^{-v}) \\ &\leq \sum_{\substack{v \subseteq \{1, \ldots, s\} \\ v \neq \varnothing}} \int_{[a^v, b^v]} \left| \frac{\partial^{|v|}}{\prod_{j \in v} \partial x_j}(g \circ h)(x^v; b^{-v}) \right| dx^v . \end{aligned}$$

As the dimension s is finite, the sum in the definition of the Hardy–Krause variation has finitely many summands and therefore it suffices to show for each summand that it is bounded.

Applying the multivariate Faà di Bruno formula (Proposition 6.2.3) to each summand yields

$$\int_{[a^v,b^v]} \left| \frac{\partial^{|v|}}{\prod_{j\in v}\partial x_j}(g\circ h)(x^v;b^{-v}) \right| dx^v$$
$$= \int_{[a^v,b^v]} \left| \sum_{\pi} \left(g^{(|\pi|)}\circ h\right)(x^v;b^{-v}) \cdot \prod_{B\in\pi} \frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v;b^{-v}) \right| dx^v$$

with the sum taken over all partitions π of the subset v. Now applying the triangle inequality gives

$$\leq \sum_{\pi} \int_{[a^v,b^v]} \left| \left(g^{(|\pi|)}\circ h\right)(x^v;b^{-v}) \cdot \prod_{B\in\pi} \frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v;b^{-v}) \right| dx^v .$$

Again, the sum has finitely many summands as there are finitely many partitions π of the finite set $v \subseteq \{1,\dots,s\}$. Thus, to show that the Hardy–Krause variation of the composition $g\circ h$ is bounded, it remains to show that each summand

$$\int_{[a^v,b^v]} \left| \underbrace{\left(g^{(|\pi|)}\circ h\right)(x^v;b^{-v})}_{(*)} \cdot \prod_{B\in\pi} \frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v;b^{-v}) \right| dx^v$$

is bounded.

From the boundedness of h given by $\widehat{V}_{\mathrm{HK}}(h) < \infty$ and $g \in C^s$, it follows that $g\circ h$ and thus the term $(*)$ is bounded on the whole integration domain $[a,b]$. Therefore, it suffices to show that

$$\int_{[a^v,b^v]} \prod_{B\in\pi} \left| \frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v;b^{-v}) \right| dx^v < \infty. \tag{6.3}$$

Following the assumption that all factors $\frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v;b^{-v})$ for all $B\in\pi$ exist and are continuous, $\frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v;b^{-v}) \in L^1([a^v,b^v])$ holds. Thus, by the generalized Hölder inequality, see for example [2, Corollary 2.11.5] applied to Equation (6.3) and using the known relation $L^p([a^v,b^v]) \subseteq L^1(a^v,b^v)$ for $p \geq 1$, it suffices to show that

$$\int_{[a^v,b^v]} \left| \frac{\partial^{|B|}}{\prod_{j\in B}\partial x_j} h(x^v;b^{-v}) \right| dx^v < \infty. \tag{6.4}$$

for all $B\in\pi$.

Recalling the definition of $\widehat{V}_{\mathrm{HK}}(h)$ from Remark 6.1.3 and with the assumption that $\widehat{V}_{\mathrm{HK}}(h) < \infty$ one gets:

$$\widehat{V}_{\mathrm{HK}}(h) = \sum_{\substack{v\subseteq\{1,\dots,s\}\\ v\neq\emptyset}} \int_{[a^v,b^v]} \left| \frac{\partial^{|v|}}{\prod_{j\in u}\partial x_j} h(x^v; b^{-v}) \right| dx^v < \infty. \tag{6.5}$$

Consequently, all summands in (6.5) are bounded. Thus, Equation (6.4) holds for all $B \in \pi$ and consequently, the Hardy–Krause variation of the composition $g \circ h$ is bounded. □

Now we can prove the theorem.

Proof (Theorem 6.2.2).
First, a mollifer function $\eta_\delta \in C^s(\mathbb{R}, \mathbb{R}_+)$ fulfilling

$$\max\left\{\left\| |a| - \eta_\delta(a) \right\|_\infty\right\} \leq \delta,$$

is chosen. Mollifying the absolute value function in a neighborhood of the origin, guarantees the existence of such a function.

The following auxiliary functions are defined:

$$\mathcal{E}_G^\delta := \int_X \eta_\delta(f(x) - \hat{f}(x))$$
$$\mathcal{E}_T^\delta := \frac{1}{N}\sum_{n=1}^{N} \eta_\delta(f(x_n) - \hat{f}(x_n))$$

Next, it is shown that the mollified error $\eta_\delta \circ (f - \hat{f})$ has bounded Hardy–Krause variation.

Since $\hat{f}$ is a composition of affine functions and the activation function $\sigma \in C^s$ by definition (5.1), $\hat{f} \in C^s([0,1]^s, \mathbb{R})$ holds. Consequently, it follows that

$$V_{\mathrm{HK}}(\hat{f}) \equiv \widehat{V}_{\mathrm{HK}}(\hat{f}) < \infty.$$

By the equation (6.2) and from the given assumption $\widehat{V}_{\mathrm{HK}}(f) < \infty$, it follows that

$$\widehat{V}_{\mathrm{HK}}(f - \hat{f}) \leq \infty \tag{6.6}$$

Using the assumptions on f and Equation (6.6), Corollary 6.2.4 can be applied to $\eta_\delta \circ (f - \hat{f})$ which yields

$$V_{\mathrm{HK}}\big(\eta_\delta \circ (f - \hat{f})\big) \leq \widehat{V}_{\mathrm{HK}}\big(\eta_\delta \circ (f - \hat{f})\big) \leq C_1 < \infty.$$

for a constanct C_1. Therefore, one can apply the Koksma-Hlawka inequality (Theorem 3.2.1) to $f - \hat{f}$ to get

$$|\mathcal{E}_G^\delta - \mathcal{E}_T^\delta| \leq C_2 \cdot \frac{V_{\mathrm{HK}}(\eta_\delta(f - \hat{f}))(\log N)^s}{N}. \tag{6.7}$$

With $C = C_1 \cdot C_2$, where the constant C_1 depends on δ and the dimension s, it follows that

$$|\mathcal{E}_G^\delta - \mathcal{E}_T^\delta| \leq C \cdot \frac{(\log N)^s}{N}. \tag{6.8}$$

By construction of η_δ it holds that

$$\left|\mathcal{E}_G - \mathcal{E}_G^\delta\right| \leq \delta \qquad \text{and} \qquad \left|\mathcal{E}_T - \mathcal{E}_T^\delta\right| \leq \delta. \tag{6.9}$$

By applying the triangle inequality to the initial generalization gap, it follows that

$$\begin{aligned} |\mathcal{E}_G - \mathcal{E}_T| &\leq \underbrace{|\mathcal{E}_G - \mathcal{E}_G^\delta|}_{\leq \delta} + \underbrace{|\mathcal{E}_T^\delta - \mathcal{E}_T|}_{\leq \delta} + |\mathcal{E}_G^\delta - \mathcal{E}_T^\delta| \\ &\leq 2\delta + |\mathcal{E}_G^\delta - \mathcal{E}_T^\delta| \\ &\leq 2\delta + C \cdot \frac{(\log N)^s}{N}. \end{aligned}$$

The bound on the generalization error follows as stated in the theorem:

$$\mathcal{E}_G \leq \mathcal{E}_T + C \cdot \frac{(\log N)^s}{N} + 2\delta. \tag{6.10}$$

□

Remark 6.2.5.
Theorem 6.2.2 gives an upper bound on the generalization error. However, the bound requires $\widehat{V}_{\mathrm{HK}}(f) < \infty$. If $f \in C^s([0,1]^s, \mathbb{R})$, this is necessarily fulfilled. Nevertheless, $f \in C^s([0,1]^s, \mathbb{R})$ is a stronger assumption on the ground truth function f than $\widehat{V}_{\mathrm{HK}}(f) < \infty$. The boundedness of the Hardy–Krause variation of f is only on the mixed-first-order derivatives of f and not on all partial derivatives up to order s as required by $f \in C^s([0,1]^s, \mathbb{R})$.

Chapter 7

Empirical Evaluation and Discussion

In the last chapter, the Koksma–Hlawka inequality was applied to the function approximation problem via Deep Neural Networks (DNNs). These theoretical results, yielded a significant improvement in the approximation of the generalization error for training sets sampled from low-discrepancy sequences compared to pseudo-random sampling.

In this chapter, these theoretical results are empirically evaluated for different examples.

7.1 Implementation details of the DNN surrogate training

Before applying the Deep Learning algorithm to optimize the weights θ of the Deep Neural Networks for the approximation different of different ground truth functions f later in this chapter, some implementation details are provided in the following. The ground truth functions studied in this chapter are the smooth sum of sines function on multiple dimensions and the approximate Ordinary Differential Equation (ODE) solution to the distance achieved by projectile motion. The implementation is done in Python using the *PyTorch* library [22] and accessible via GitHub[1].

As the main goal of this chapter is to evaluate the impact of different sampling strategies, the Deep Learning algorithm was applied on training sets sampled from three different strategies: pseudo-random sampling and two low-discrepancy sequences: Sobol' and Halton as introduced in Chapter 3. Each training set is obtained by evaluating the ground truth function f at the sampled points. In the experiments, the number of training samples N is varied from 64 up to 8192. According to the training set, a test set $\mathcal{T}$ is chosen from the remaining sampling strategies. To ensure comparability and uniform coverage of the domain, the test sets are always chosen to be the full low-discrepancy sequence with $|\mathcal{T}| = 8192$ points. The setup can be summarized as in Table 7.1.

[1] https://github.com/janikvhrubant/nn-sampling-analysis

Training Set	Test Set
Pseudo-random	Sobol'
Sobol'	Halton
Halton	Sobol'

Table 7.1: Test sets used for the evaluation of the DNNs trained on different training sets.

For the evaluation of the approximation quality, the following test error metric is then applied on the test set $\mathcal{T}$

$$\text{MAE} = \frac{1}{|\mathcal{T}|} \sum_{\mathbf{x} \in \mathcal{T}} |f(\mathbf{x}) - \hat{f}_\theta(\mathbf{x})|.$$

The test set $\mathcal{T}$ is differing for the three training sets. The setup can be summarized as in Table 7.1.

Further the Deep Learning algorithm is evaluated for a varying number of epochs between 100 and 2000 together with the optimizer introduced in Chapter 6: Adam [8].

For the other training hyperparameters, a systematic study was conducted using *Optuna* [1]. Hereby the architecture of the DNNs was varied in terms of the number of hidden layers, the number of neurons per layer and the activation function. Further different batch sizes were examined. For the two optimizers, different parameters were studied. The parameter ranges are summarized in Table 7.2. The specific configurations used in the experiments will be detailed in the corresponding sections.

DNN Architecture	
Number of hidden layers	2 – 20
Number of neurons per layer	2 – 12
Activation function	Sigmoid, Tanh
Training Setup	
Batch size	64, 128, 256, 512, 1024
Adam Hyperparameters	
Learning rate	$1.0e{-}5$ – $1.0e{-}2$
Weight decay	$1.0e{-}8$ – $1.0e{-}3$
β_1	0.85 – 0.99
β_2	0.95 – 0.999
ϵ	$1e-8$

Table 7.2: Configurations used for Hyperparameter optimization.

7.2 Multi-dimensional sum of sines

As a first example, the multidimensional and nonlinear sum of sines function is studied in for different dimensions $s \in \{6, 8, 10\}$

$$f(\mathbf{x}) = \sum_{i=1}^{s} \sin(4\pi x_i), \quad \mathbf{x} \in [0,1]^s. \tag{7.1}$$

To add variance to the function, the input is scaled by a factor of 4π. It is clear, that $f \in C^\infty([0,1]^s)$ and thus $V_{\mathrm{HK}}(f) \sim \hat{V}_{\mathrm{HK}}(f) < \infty$ on the compact domain $[0,1]^s$.

10 dimensional sum of sines

Now, the Deep Learning algorithm is applied to approximate the Equation (7.1) for $s = 10$ dimensions.

The Deep Learning algorithm is applied with the following configurations obtained by hyperparameter optimization as described in Section 7.1:

Adam Training Setup	
DNN Architecture	
Number of hidden layers	9
Number of neurons per layer	12
Activation function	Sigmoid
Training Setup	
Batch size	256
Adam Parameters	
Learning Rate	$5.426e{-3}$
Weight Decay	$4.357e{-5}$
β_1	0.92
β_2	0.959
ε	$1e-8$

Table 7.3: DNN and Adam configurations for the 10D sum of sines.

Lion Training Setup	
DNN Architecture	
Number of hidden layers	7
Number of neurons per layer	12
Activation function	Sigmoid
Training Setup	
Batch size	64
Lion Parameters	
Learning Rate	$5.565e{-4}$
Weight Decay	$2.105e{-5}$
β_1	0.91
β_2	0.965

Table 7.4: DNN and Lion configurations for the 10D sum of sines.

The according DNNs are trained on training sets with $N \in \{128, 256, 512, 1024, 2048, 4096, 8192\}$ samples drawn from pseudo-random, Sobol' and Halton sequences.

Hier kommt ein Plot für alle 6 Kombinationen: Adam/Lion und Sobol/Halton/Pseudo-random

1. Test Error Entwicklung über Epochs (für alle 6 Kombinationen) + Train Error Entwicklung über Epochs (für alle 6 Kombinationen)

2. Test Error Entwicklung über N (für alle 6 Kombinationen) + Train Error Entwicklung über N (für alle 6 Kombinationen)

7.3 Projectile Motion

As a second illustrative example, this section investigates the motion of a projectile. The experiment is introduced by formulating the underlying ODEs system that describes the physical dynamics of the process. Based on approximate solutions of these ODEs at the prescribed training points, DNNs are trained and subsequently evaluated.

7.3.1 ODE formulation of projectile motion

Projectile motion describes the trajectory of an object launched into the air and evolving under gravity and aerodynamic drag. We consider a planar motion with state

$$x(t) \in \mathbb{R} \quad \text{(horizontal position)}, \qquad h(t) \in \mathbb{R} \quad \text{(vertical position)},$$

and velocities $u(t) = \dot{x}(t)$, $w(t) = \dot{h}(t)$, according to the system parameters θ (defined below), including the launch angle α. The motion is terminated at the first impact time

$$t_\star \;=\; \inf\{t > 0 \,:\, h(t) = 0\},$$

and the traveled range is

$$R(\theta) \;=\; x(t_\star). \tag{7.2}$$

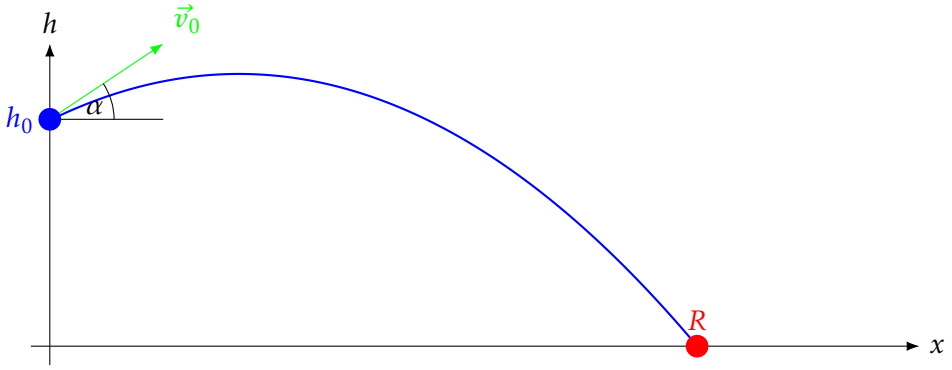


FIGURE 7.1: Projectile motion setup with projectile launched from height h_0 and initial velocity $\vec{v}_0$ at angle α to the horizontal. The final range R is achieved at impact time $t_\star$.

Parameters and modeling assumptions

1. The moving object is modeled as a spherical projectile with mass m in the range $0.3 - 0.78\,\text{kg}$ and radius r in range $0.0213 - 0.124\,\text{m}$. The extreme values correspond to a golf ball of gold and a basketball, respectively.
2. Gravitational acceleration is constant, $g = 9.81\,\text{m/s}^2$.
3. Air density ρ is varying in the range of $0.269 - 1.225\,\frac{\text{kg}}{\text{m}^3}$ which corresponds to the air density at the Mount Everest summit and at sea level, respectively.
4. The drag coefficient c_d of the projectile is constant in the range $0.25 - 0.7$.
5. The initial hight h_0 of the projectile is in the range $1 - 100\,\text{m}$.
6. The initial speed v_0 of the projectile is in the range $1 - 50\text{m/s}$.
7. The launch angle α of the projectile is in the range $0 - 90°$.

Quadratic-drag model

Let $\mathbf{z}(t) = (x(t), h(t), u(t), w(t))^\top$ with horizontal/vertical positions (x, h) and velocities $(u, w) = (\dot{x}, \dot{h})$. Denote $\mathbf{v} = (u, w)$ and $\|\mathbf{v}\| = \sqrt{u^2 + w^2}$. For a spherical body of radius r and mass m in air of density ρ with constant drag coefficient c_d, the cross-sectional area is $A = \pi r^2$ and the drag-specific constant is

$$k = \frac{\rho c_d A}{2m} = \frac{\rho c_d \pi r^2}{2m}.$$

The physically consistent planar model with quadratic aerodynamic drag reads

$$\dot{\mathbf{z}}(t) = \begin{pmatrix} u \\ w \\ -k\|\mathbf{v}\| u \\ -g - k\|\mathbf{v}\| w \end{pmatrix}. \tag{7.3}$$

Equivalently, in component form,

$$\dot{x} = u, \quad \dot{h} = w, \quad \dot{u} = -k\|\mathbf{v}\| u, \quad \dot{w} = -g - k\|\mathbf{v}\| w. \tag{7.4}$$

Initial conditions
Given launch speed $v_0 > 0$, angle $\alpha \in [0, \frac{\pi}{2}]$ and height $h_0 \geq 0$,

$$x(0) = 0, \qquad h(0) = h_0, \qquad u(0) = v_0 \cos\alpha, \qquad w(0) = v_0 \sin\alpha. \tag{7.5}$$

Parameterization for learning
We collect the physical parameters in

$$\theta = (\rho, r, c_d, m, h_0, \alpha, v_0) \in \Theta$$

with the given application-driven bounds:

$$\begin{aligned} &\rho \in [\rho_{\min}, \rho_{\max}] = [0.269, 1.225]\ \text{kg/m}^3, \quad r \in [r_{\min}, r_{\max}] = [0.0213, 0.124]\ \text{m}, \\ &c_d \in [c_{d,\min}, c_{d,\max}] = [0.25, 0.70], \quad m \in [m_{\min}, m_{\max}] = [0.30, 0.78]\ \text{kg}, \\ &h_0 \in [h_{0,\min}, h_{0,\max}] = [1, 100]\ \text{m}, \quad \alpha \in [0, \frac{\pi}{2}], \quad v_0 \in [v_{0,\min}, v_{0,\max}] = [1, 50]\ \text{m/s}. \end{aligned}$$

Here, Θ denotes the Cartesian product of all admissible intervals from above. In the experiments, $\mathbf{y} \in [0,1]^7$ is sampled according to the chosen sampling method. Then, $\theta(\mathbf{y})$ is mapped affinely to the physical parameter ranges via

$$p(\mathbf{y}) = p_{\min} + (p_{\max} - p_{\min})\, y, \quad p \in \{\rho, r, c_d, m, h_0, \alpha, v_0\},$$

and define the parameter-to-output map

$$\mathcal{F} : [0,1]^7 \to \mathbb{R}, \qquad \mathcal{F}(\mathbf{y}) = R\big(\theta(\mathbf{y})\big), \tag{7.6}$$

where $R(\cdot)$ is given by (7.2) for the ODE (7.3)-(7.5). The DNNs are trained to approximate $\mathcal{F}$.

Well-posedness
The right-hand side in (7.3) is continuous and locally Lipschitz in $\mathbf{z}$ (the map $\mathbf{v} \mapsto \|\mathbf{v}\|\mathbf{v}$ is locally Lipschitz), hence for any fixed $\theta \in \Theta$ there exists a unique maximal solution up to $t_\star$. Moreover $h(t)$ is strictly decreasing after the apex, ensuring a unique impact time.

Numerical solution
We integrate (7.3) with a fixed-step explicit scheme. By a small step size $\Delta t > 0$, updates are computed with the forward Euler method

$$\begin{pmatrix} x_{n+1} \\ h_{n+1} \\ u_{n+1} \\ w_{n+1} \end{pmatrix} = \begin{pmatrix} x_n \\ h_n \\ u_n \\ w_n \end{pmatrix} + \Delta t \begin{pmatrix} u_n \\ w_n \\ -k\|\mathbf{v}_n\| u_n \\ -g - k\|\mathbf{v}_n\| w_n \end{pmatrix}.$$

and terminate at the first n with $h_{n+1} \le 0$. This approximates the traveled range $R \approx x_{n+1}$.

7.3.2 DNN training and evaluation

Since $R \circ \theta \in C^1([0,1]^7)$, we have $\widehat{V}_{\mathrm{HK}}(R \circ \theta) < \infty$. For small enough step size Δt, the numerical approximation of $R \circ \theta$

TODO: Argument why the numerical approx. is also in C^1?

The Deep Learning algorithm is applied with the following configurations obtained by hyperparameter optimization as described in Section 7.1:

DNN Architecture	
Number of hidden layers	8
Number of neurons per layer	8
Activation function	Tanh
Training Setup	
Batch size	256
Number of epochs	2000
Optimizers	
Adam	
Lion	
Hyperparameters	
Optimized via	Optuna

Part III

X-ray Simulation using QMC Methods

Chapter 8

Motivation and Problem Statement

8.1 Computed Tomography Imaging

Computed Tomography (CT) imaging is a widely used diagnostic technique that generates detailed cross-sectional images by combining multiple X-ray projections acquired from different angles. An X-ray source rotates around the patient while a detector array records the transmitted radiation. Each projection reflects the cumulative attenuation of X-rays as they traverse tissues of varying density and composition.

Reconstruction algorithms, such as filtered backprojection or iterative methods, convert these projections into volumetric images, which can be further processed to enhance contrast and reduce noise. A key challenge is scattered radiation (Compton and Rayleigh scattering), which introduces artifacts and degrades image quality.

The goal of *scatter reduction in CT* is therefore to suppress scatter contributions in the acquired two-dimensional projection data, thereby improving the accuracy of the reconstructed three-dimensional volume.

8.2 X-ray Imaging and the Challenge of Scatter

X-ray CT relies on measuring the attenuation of X-rays along straight-line paths through a phantom. The photon trajectories typically extend from an X-ray source $\mathcal{S}$ to a detector element $\mathcal{D}$, forming the path:

$$l = \overrightarrow{\mathcal{SD}}$$

Under idealized conditions, the attenuation process is governed by the *Lambert-Beer law*, which describes an exponential decay in intensity I as the beam interacts with the material. As stated in [17, Chap. 7], this relationship is given by:

$$I = I_0(E) \cdot \int_0^{E_{\max}} \exp\left(- \int_{\mathcal{S}}^{\mathcal{D}} \mu(x,E)\, dx \right) dE \tag{8.1}$$

Here, $I_0(E)$ denotes the incident intensity of X-rays with initial energy E at the source $\mathcal{S}$. $\mu(x,E)$ represents the linear attenuation coefficient at spatial location x along the path l, for energy E. The attenuation coefficient is representing the attenuation per unit length and has usually the unit $\frac{1}{cm}$. The resulting intensity I is measured at the detector element $\mathcal{D}$.

Although exponential attenuation along straight-line paths such as $\overrightarrow{SD}$ is the idealized model for X-ray imaging, this assumption is systematically violated in practice. As X-rays traverse the scanned phantom, they undergo scattering interactions - such as Compton or Rayleigh scattering - that alter their trajectories. Despite deviating from the primary path, these scattered X-rays may still reach the detector, adding unintended signals at the detector. As a result, the measured intensities no longer represent pure line integrals of the attenuation map. This discrepancy introduces nonlinear errors and visible artifacts in the reconstructed image, ultimately degrading both visual quality and quantitative accuracy.

Scattered radiation is a major source of image artifacts — such as cupping and streaks — and reduces both spatial and contrast resolution. These effects not only degrade visual image quality but also compromise the accuracy of quantitative measurements. But precise CT-images are important for various clinical applications, such as treatment planning in radiation therapy. Generating accurate three-dimensional models of the phantom is crucial for more precise dose calculations, strong enough to be effective but sufficiently weak to minimize unintended tissue damage.

In clinical practice, such three-dimensional patient models are represented in Hounsfield Units (HUs), which encode tissue-specific attenuation properties and serve as the basis for accurate dose calculation in radiation therapy [17, Chapter 8].

In CT, the reconstruction algorithm produces a volumetric map of effective attenuation coefficients, from which each voxel value is normalized according to the Hounsfield definition. The resulting HU values form the basis of the displayed grayscale intensities in the CT image.

HU provide a dimensionless measure of X-ray attenuation. They are defined by normalizing the reconstructed attenuation coefficient μ with respect to water and air:

$$\mathrm{HU} = 1000 \cdot \frac{\mu - \mu_{\mathrm{water}}}{\mu_{\mathrm{water}} - \mu_{\mathrm{air}}} \approx 1000 \cdot \frac{\mu - \mu_{\mathrm{water}}}{\mu_{\mathrm{water}}}, \tag{8.2}$$

where μ_{water} and μ_{air} denote the attenuation coefficients of water and air, respectively. By convention, water is assigned 0 HU and air -1000 HU, while dense bone can exceed $+1000$ HU. Although HU reduce scanner- and protocol-dependent variability, they still depend on factors such as tube voltage and reconstruction settings. They are therefore best understood as a standardized but not absolute measure of tissue attenuation in clinical CT imaging.

The impact of scatter becomes especially pronounced in modern CT systems using high-energy X-rays or large-area flat-panel detectors, where scatter may dominate the measured signal. Accurate modeling and correction of scatter are thus essential for achieving high-fidelity CT images, particularly in clinical applications where precision and reliability are paramount [17].

For precise estimation of the models and HU values respectively, it is therefore important to correct for scatter effects in the acquired projection data.

8.3 Scatter Correction Methods for X-ray Imaging

8.3.1 Overview of Scatter Correction Techniques

In X-ray imaging, scattered photons are a major source of image artifacts and quantitative inaccuracies. To mitigate these effects, a range of computational scatter correction methods has been developed. These approaches can be broadly classified into the following categories:

- **Empirical and Analytical Methods:**
  These include techniques such as primary modulation, convolution-based correction, and energy windowing. Scatter is typically estimated using simplified models or empirical kernels, often assuming a smooth background distribution, and then subtracted from the measured signal. While computationally efficient, these methods rely on assumptions regarding the spatial and energy distribution of scattered photons. As a result, they may fail to accurately model complex scatter phenomena in heterogeneous anatomical structures.
- **Physics-Based Models:**
  These methods aim to provide a more accurate representation of the underlying photon transport physics, including scattering phenomena. Among them, MC simulation is regarded as the most rigorous and comprehensive technique due to its ability to statistically model complex photon interactions without relying on simplifying assumptions. In such simulations, primary and scattered photon contributions are detected separately, allowing the estimated scatter signal to be subtracted from measured data in order to restore image fidelity.

In 2011, Sisniega et al. [27] already demonstrated promising results[27] using computationally expensive MC simulations of CT scans of a cylindrical phantom with two bone inserts. The results showed significant improvements in image quality. The presented QMC simulation mthod aims to further enhance the efficiency of such simulations and yields the following results for a similar phantom, once with scattering and once without scattering. In Figure 8.1, the Sobol' sampling method was used and $5e{+}6$ photons were simulated from a spectrum with a maximum energy of 35 keV which corresponds to a 35 kVp X-ray tube voltage.

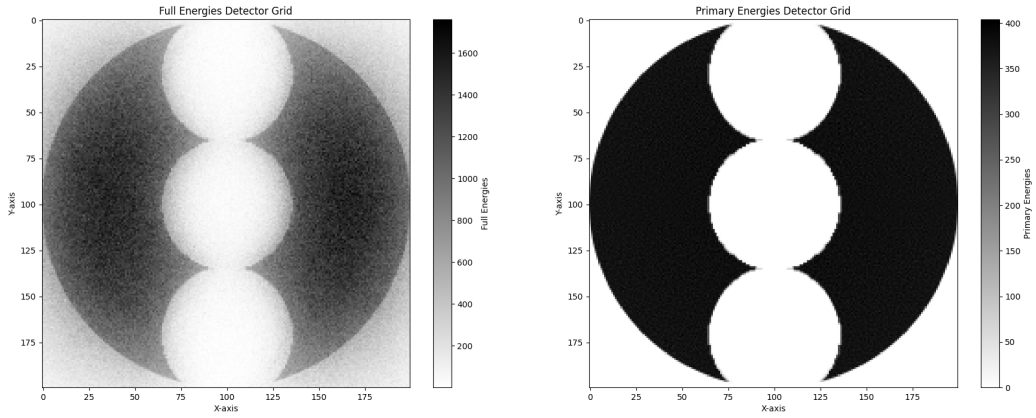


FIGURE 8.1: QMC generated Cone-Beam X-ray image with $5e{+}6$ photons of a 70 mm soft-tissue cylinder with three bone inserts. Left: Simulation of a X-ray image with Compton scattering. Right: Image after removing Compton scatters.

8.3.2 Monte Carlo Simulation

To achieve scatter correction using MC simulation, a three-dimensional phantom is first estimated by analyzing the X-ray projections. Then, an MC simulation is performed to estimate the scatter signal $I_{\text{scattered}}$ for each detector pixel. This estimated scatter signal is then subtracted from the measured intensity I_{measured}:

$$I_{\text{corrected}} = I_{\text{measured}} - I_{\text{scattered}} \tag{8.3}$$

The MC simulation is a powerful computational technique that follows physics-based principles to model the transport of photons through matter. It is widely recognized as the gold standard for scatter correction in CT and related imaging modalities. This status is attributed to several key factors:

- **Physical Accuracy:**
  MC methods simulate the stochastic nature of photon interactions - including Compton and Rayleigh scattering, photoelectric absorption, and multiple scattering events - based on fundamental physical cross-sections and material properties.
- **Comprehensive Modeling:**
  Unlike analytical or empirical methods, MC simulations can account for complex geometries, heterogeneous materials and realistic X-ray spectra, providing highly accurate estimates of the scatter signal.
- **Validation Benchmark:**
  Due to their accuracy, MC-based scatter estimates are routinely used as reference standards for validating and benchmarking faster, approximate correction methods.

8.4 High-Level Overview of the Monte Carlo Simulation

The MC simulation of photon transport for scatter correction typically involves the following key steps [27]:

1. **Photon Emission:**
   Photons are emitted from a virtual X-ray source, with their energies sampled from a predefined source spectrum.
2. **Photon Tracking:**
   Each photon is tracked as it propagates through the object. At every step, the probability of interaction (either scattering or absorption) is determined by the local material properties and corresponding interaction cross-sections.
3. **Interaction Sampling:**
   Upon interaction, the event type (e.g., Compton scattering, Rayleigh scattering, or photoelectric absorption) and the resulting change in photon direction and energy are sampled from the relevant probability distributions.
4. **Detection:**
   Photons that reach the detector—either unscattered or after one or more scattering events—are recorded. The simulation distinguishes between primary and scattered photons, thereby enabling an accurate estimation of the scatter contribution at each detector element.
5. **Statistical Averaging:**
   By simulating a large number of photons, the method builds a statistically robust estimate of the scatter distribution. The accuracy of the result increases with the number of simulated photons and ultimately converges toward a stable intensity distribution.

The schematic flow of the Monte Carlo simulation process is summarized in Figure 8.2.

Monte Carlo Photon Transport Algorithm

- **Initialization:** Define geometry, material composition, and source spectrum.
- **Loop over photons:**
  - Sample the step size to the next interaction point.
  - Move the photon; check for boundary crossings.
  - Sample the interaction type; update direction and energy.
  - If the photon reaches the detector, record the event.
  - If the photon is absorbed or exits the system, terminate its trajectory.
- **Aggregation:** Compute both scatter and primary intensity distributions.

FIGURE 8.2: Pseudocode representation of the Monte Carlo algorithm for photon transport in scatter modeling.

8.5 Monte Carlo Methods: Benefits & Drawbacks

MC simulations are widely regarded as the gold standard for scatter correction in computed tomography due to their ability to model photon-matter interactions from first principles. These simulations accurately reproduce all relevant physical scattering phenomena - including Compton and Rayleigh scattering - and can accommodate arbitrarily complex geometries and heterogeneous material compositions. Owing to this high degree of physical fidelity, MC-based methods yield highly accurate scatter estimates and are commonly employed as reference standards for evaluating and validating alternative correction approaches.

However, the high accuracy of Monte Carlo simulations comes at the cost of substantial computational effort. Producing low-noise scatter estimates requires the simulation of a large number of photon histories to ensure statistical convergence. Consequently, the runtime scales with the desired level of accuracy and may range from several minutes to multiple hours or even days, depending on the complexity of the scene and the available computational resources. This computational burden poses a major limitation, particularly in applications involving large datasets or iterative reconstruction workflows.

To address the computational inefficiency of slow-converging MC methods, recent research has explored strategies to accelerate physically accurate photon transport simulations. Among these, the application of QMC methods has gained increasing attention. By replacing random sampling with deterministic low-discrepancy sequences, QMC techniques can achieve significantly faster convergence while maintaining comparable accuracy. Recent QMC-based scatter correction algorithms have demonstrated runtime reductions of multiple orders of magnitude, thereby enabling high-fidelity simulations even in time-sensitive or resource-constrained environments [15, 16, 7].

Chapter 9

Physical laws of Photon Simulation

This chapter introduces the physical and mathematical principles underlying the simulation of X-ray imaging. Central to this is the modeling of individual photon interactions with matter, including attenuation and scattering processes. These interactions are inherently stochastic due to the quantum nature of radiation-matter interactions. The stochastic nature of photons and their interactions with matter is modelled using monte Carlo methods of uniformly distributed values between 0 and 1, which are then transformed to follow the physical probability distributions relevant to the specific interactions being simulated. This approach allows for a realistic representation of photon transport and interaction within the imaging system.

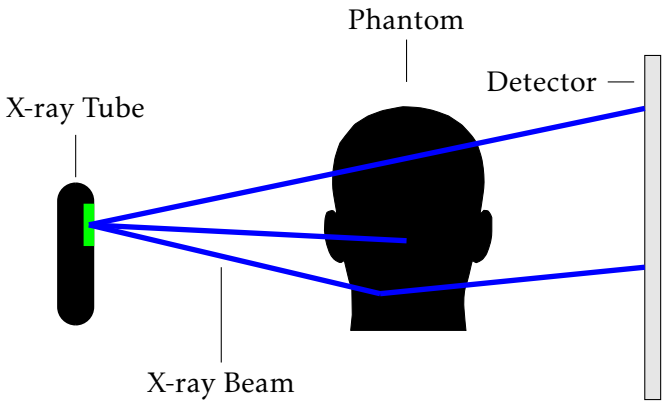


Figure 9.1: Schematic illustration of photon transport in X-ray imaging.

In a typical simulation workflow, individual photons are emitted from the X-ray tube and propagate through air before reaching the phantom. Upon entering the phantom, they may interact with the material through scattering or absorption, depending on the local composition of the tissue and photon energy. Only a fraction of the photons will exit the phantom, continue their path through air, and ultimately reach the detector, contributing to the final image formation.

The simulation framework described here forms the basis for all subsequent analyses presented in this thesis.

9.1 X-Ray Tube Spectrum

X-ray beams are generated within evacuated glass tubes containing several critical components that convert electrical energy into X-ray photons and heat. The X-ray tube acts as an energy converter, where the vast majority of the input energy is transformed into heat and only a small fraction becomes useful radiation. The following Figure 9.2 illustrates the basic construction of an X-ray tube as in [24]

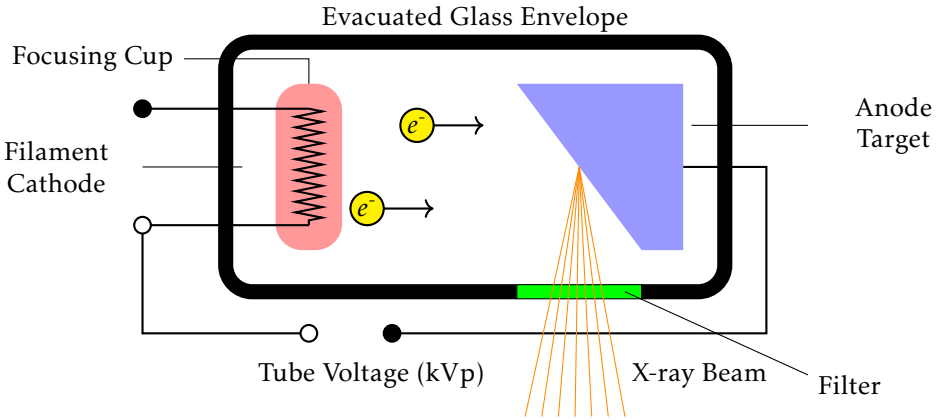


FIGURE 9.2: Schematic of an X-ray tube with Filter.

To capture this randomness, Monte Carlo methods are employed. In such simulations, random numbers—typically sampled uniformly from the interval $[0,1]$ are transformed into physically meaningful quantities according to the relevant probability distributions. This probabilistic modeling enables realistic simulation of photon transport and forms the basis for the analyses presented in this thesis.

The key components are:

- **Filament (Cathode):** A tungsten wire filament serves as the electron source through thermionic emission. When a current of approximately 3-6 A passes through it, the filament reaches incandescence, releasing electrons from its surface. These free electrons form a cloud near the cathode until they are accelerated toward the anode by the applied high voltage.
- **Focusing Cup:** The filament is embedded in a negatively charged, nickel-made focusing cup. Its function is to electrostatically shape and direct the electron stream toward the anode's focal spot, thereby influencing the resolution and size of the resulting X-ray beam.
- **Anode Target:** The anode consists of a tungsten target, often embedded in a copper support. Tungsten is used for its high atomic number and melting point, enhancing X-ray production and durability. The copper base improves heat dissipation. Typically, less than 1% of the electron energy is converted into X-rays, with the remainder generating heat that must be effectively managed.
- **Evacuated Glass Envelope:** All components are sealed within a borosilicate glass or metal-ceramic housing evacuated to a low pressure (typically 10^{-5} to 10^{-7} hPa). The vacuum allows unimpeded electron flow and prevents arcing.

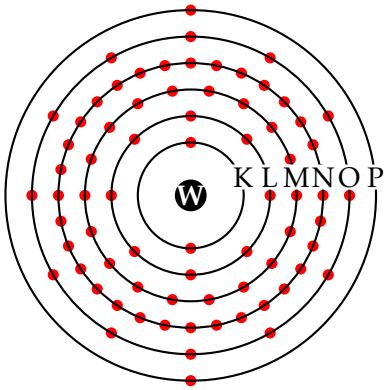
The housing is usually immersed in insulating oil to provide thermal and electrical isolation.

- **Filter (Filtration):** A filter is positioned at the tube exit, between the anode target and the patient, to selectively absorb low-energy photons. These photons contribute little to image formation but significantly increase patient dose. Their removal (beam hardening) improves image quality by reducing scatter and enhancing contrast. Common filter materials are aluminum, copper and molybdenum; in this thesis, a clinical setup with 1 mm aluminum (Al) and 0.2 mm copper (Cu) is used [24, 29].

When high-energy electrons strike the tungsten target, X-ray photons are produced through two primary mechanisms:

- **Bremsstrahlung (Braking Radiation):** This accounts for approximately 80% of X-ray production. When electrons pass close to tungsten nuclei, they are decelerated by the electrostatic attraction, causing them to lose kinetic energy that is emitted as X-ray photons. This process produces a continuous spectrum of X-ray energies from near zero up to the maximum electron energy.
- **Characteristic Radiation:** When high-energy electrons eject inner-shell electrons from tungsten atoms, vacancies are created in the K or L shells. These vacancies are subsequently filled by electrons from outer shells and the resulting energy difference is emitted as an X-ray photon with a discrete, element-specific energy. The corresponding peaks in the spectrum therefore occur at the differences of the binding energies of the involved shells. For reference, the atomic model of tungsten is shown in Figure 9.3a and the approximate binding energies of its shells and subshells are listed in Table 9.3b.

  From these binding energies, the main characteristic line energies can be derived. For example, the $K\alpha_1$ line corresponds to an electron transition from the L_3 subshell into the K-shell (69.5–10.2 keV $\approx$ 59.3 keV), the $K\alpha_2$ line results from the $L_2 \rightarrow$ K transition (69.5–11.5 keV $\approx$ 58.0 keV) and the $K\beta_1$ line originates from the $M_2/M_3 \rightarrow$ K transition (69.5–2.3 keV $\approx$ 67.2 keV). These discrete transitions explain the multiple sharp peaks superimposed on the continuous bremsstrahlung spectrum. In practice, only the K, L and M shells are relevant for X-ray production, while binding energies of the N shell and higher are too small to contribute significantly.



(A) Bohr model of tungsten (W, Z=74) with electron shells.

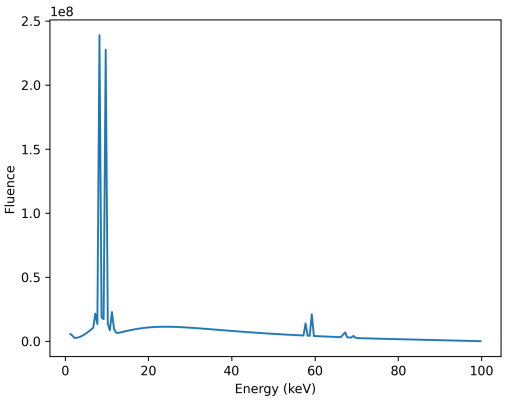
Shell / Subshell	Binding Energy (keV)
K	69.5
L_1	12.1
L_2	11.5
L_3	10.2
M_1	2.82
M_2	2.30
M_3	2.15
N_1	0.43
N_2	0.32
N_3	0.22

(B) Approximate binding energies of tungsten shells/subshells.

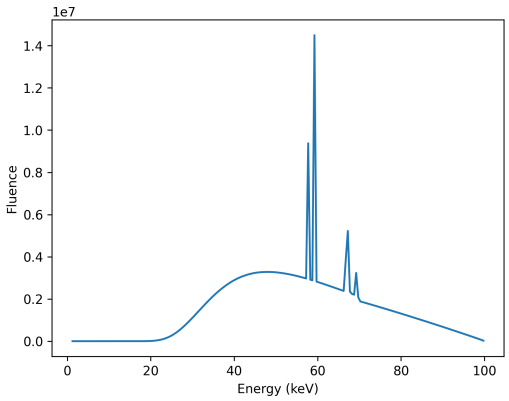
FIGURE 9.3: (a) Tungsten atom schematic and (b) corresponding binding energies.

Although it is not necessary to understand every technical detail of the X-ray apparatus for the purposes of simulation, this background information allows one to assess how key simulation parameters influence the resulting X-ray spectrum and photon behavior.

Figure 9.4a below shows an resulting energy spectrum for an X-ray tube with a tungsten cathode operated at 100 kVp. It illustrates the resulting distribution of photon energies, which is shaped by both the material and the applied voltage. The intensity of the different photon energies is measured in *spectral fluence* in $\mathrm{cm}^{-2}\,\mathrm{keV}^{-1}$, which describes the number of X-ray photons per unit area per unit energy interval. It certainly shows the two components of the X-ray spectrum: the continuous bremsstrahlung spectrum and the characteristic radiation peaks:



(A) Without additional filter.
Features: continuous bremsstrahlung up to 100 kVp; pronounced L-lines (~8-12 keV) and K-lines (~59-67 keV).



(B) With filter (1 mm Al + 0.2 mm Cu).
Effect: removal of low-energy photons (beam hardening); suppressed L-lines; continuum shifted toward higher mean energy.

FIGURE 9.4: X-ray spectra for a tungsten anode at 100 kVp: (a) without and (b) with added filtration. The filter selectively attenuates low-energy photons, reducing patient dose while preserving higher-energy components. Both spectra were generated using *SpekPy* [23].

The applied fiter in Figure 9.4b effectively removes low-energy photons below approximately 30 keV, which would otherwise contribute to patient dose without enhancing image quality, as low energy photons are likely to be absorbed superficially. The filter thus "hardens" the beam by increasing its mean energy, which improves penetration and reduces scatter. The characteristic K-lines remain prominent, even though their intensity is slightly reduced due to the filter's attenuation.

9.2 Photon Generation

In the context of Monte Carlo simulations, photons are generated with initial positions, directions and energies that reflect the physical properties of the X-ray source and the imaging setup.

9.2.1 Photon Energy

The photon energy is sampled according to the X-ray tube spectrum as described in Section 9.1. As in Figure 9.4, the spectrum maps the photon energy to the according frequency. In the simulation applied in this thesis, the spectra are generated utilizing *SpekPy* [23, 25].

The actual energies of the emitted photons in the simulation are then determined by *inverse transform sampling* [19, Chapter 4]. To determine a matching random variable with the desired distribution, the following steps need to be made:

(i) Construct the appropriate probability density function from the given spectrum.

(ii) Compute the Cumulative Distribution Function (CDF) from the probability density function F.

(iii) Determine the generalized inverse of the CDF F^{-1}.

(iv) Sample a uniformly distributed random variable U on the interval $[0,1]$ and compute the photon energy as $E = F^{-1}(U)$.

To get the probability density function from the given spectrum, the fluences w_i of the energies e_i in the spectrum are normalized such that

$$p_i = \frac{w_i}{\sum_j w_j}.$$

Now, the CDF $F : \mathbb{R} \to [0,1]$ can be computed as

$$F(e_i) = \mathbb{P}[E \leq e_i] = \sum_{j \leq i} p_j .$$

It is clear that $F(e_i)$ is monotonically increasing with $F(-\infty) = 0$ and $F(\infty) = 1$. The generalized inverse is defined as follows:

Definition 9.2.1 (Generalized Inverse)**.**
Let $f : \mathbb{R} \to \mathbb{R}$ be a monotonically increasing function. The *generalized inverse* $f^{-1} : \mathbb{R} \to \mathbb{R}$ is defined as

$$f^{-1}(y) = \inf\{x \in \mathbb{R} : f(x) \geq y\}.$$

As F is monotonically increasing, the generalized inverse F^{-1} exists and is also monotonically increasing. The photon energies are then sampled via

$$E = F^{-1}(U),$$

where U is a uniformly distributed random variable on the interval $[0,1]$.

The following theorem shows that the evaluation of the inverse CDF of the uniformly distributed random variable $U \sim \mathcal{U}(0,1)$ inherits the desired distribution of the spectrum.

Theorem 9.2.2.
Let $F : \mathbb{R} \to [0,1]$ be a continuous and strictly increasing cumulative distribution function and F^{-1} its generalized inverse. Then for a random variable X generated by

$$X := F^{-1}(U)$$

for $U \sim \mathcal{U}(0,1)$, it holds that X has cumulative distribution function F, i.e.,

$$\mathbb{P}[X \leq x] = F(x), \quad \text{for all } x \in \mathbb{R}.$$

Proof.
Since F is continuous and strictly increasing, its inverse F^{-1} exists. For any $x \in \mathbb{R}$, one has

$$\mathbb{P}[X \leq x] = \mathbb{P}[F^{-1}(U) \leq x] = \mathbb{P}[U \leq F(x)] = F(x),$$

because $U \sim \mathcal{U}(0,1)$ and thus $\mathbb{P}[U \leq u] = u$ for all $u \in [0,1]$. This shows that X has CDF F. □

Remark 9.2.3.
In the continuous setting with a distribution with strictly positive probability density function and thus, strictly increasing CDF, the generalized inverse coincides with the usual inverse.

9.2.2 Photon Position and Direction

In addition to the photon energy at emission, its initial position and direction must be specified. Within the simulation framework, two different source models are considered:

1. **Point Source:** Photons are emitted from a single point in space, typically representing the focal spot of the X-ray tube. The emission direction is sampled uniformly within a conical emission region defined by the half-opening angle Θ.

2. **Extended Source:** Photons are emitted from a finite rectangular area on a plane, with each photon initially traveling perpendicular to this plane.

Point Source

This model reflects the global imaging geometry of an X-ray tube with a pointlike focal spot and divergent beam. In line with common practice (e.g. Geant4's angular settings [5]), photon directions are sampled uniformly on a spherical cap, i.e. uniformly in the solid angle.

All photons are emitted from a fixed source position $\vec{p} = (p_1, p_2, p_3) \in \mathbb{R}^3$. To describe the corresponding probability measure, recall that the unit sphere S^2 in spherical coordinates is parameterized by

$$x = \sin\theta\cos\phi, \quad y = \sin\theta\sin\phi, \quad z = \cos\theta,$$

with $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$. The induced surface element on S^2 is given by

$$d\Omega = \sin\theta \, d\theta \, d\phi,$$

see for instance [28, Theorem 5-6]. A uniform distribution on the spherical surface thus corresponds to a constant density with respect to this measure. Restricting the support to the spherical cap $\{\theta \in [0, \Theta]\}$ gives the normalized density

$$f(\theta, \phi) = \frac{1}{2\pi\,(1 - \cos\Theta)} \quad \text{for } \theta \in [0, \Theta],\ \phi \in [0, 2\pi).$$

For sampling, one draws $u_1, u_2 \sim \mathcal{U}(0,1)$ and sets

$$\phi = 2\pi u_2, \qquad \cos\theta = 1 - u_1\,(1 - \cos\Theta), \qquad \theta = \arccos\Big(1 - u_1\,(1 - \cos\Theta)\Big),$$

followed by

$$\vec{d} = \Big(\sin\theta\cos\phi,\ \sin\theta\sin\phi,\ \cos\theta\Big).$$

This procedure yields a conical beam with half-opening angle Θ whose directions are uniformly distributed over the spherical cap surface. Photon energies are sampled independently from the spectrum as described above. Figure 9.5 illustrates the resulting geometry.

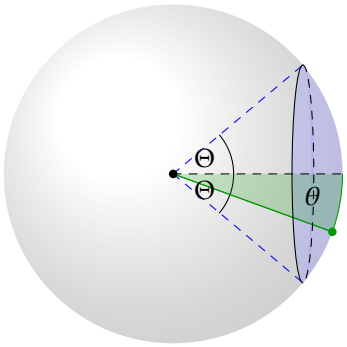


FIGURE 9.5: Sampling of photon directions uniformly within a cone defined by the half-opening angle Θ.

In Geant4's `General Particle Source`, this corresponds to *isotropic* angular sampling restricted to a cone via min/max θ and ϕ (e.g. `/gps/ang/type iso`, `/gps/ang/mintheta 0`, `/gps/ang/maxtheta Θ`, `/gps/ang/minphi 0`, `/gps/ang/maxphi 360 deg`).

Extended Source
In contrast, the finite planar source serves primarily as an analytical model. It describes a local view of the photon–phantom interaction and is employed to study the influence of source extension on scattering and attenuation effects.

The initial position of each photon is sampled uniformly on the rectangular surface of the source. For a rectangular source with vertices $P_1, P_2, P_3, P_4 \in \mathbb{R}^3$, the position of each photon is determined by two random variables $u_1, u_2 \sim \mathcal{U}(0,1)$ via bilinear interpolation:

$$\vec{p} = (1-u_1)(1-u_2)P_1 + u_1(1-u_2)P_2 + u_1u_2P_3 + (1-u_1)u_2P_4\,.$$

This yields a uniform distribution of starting points over the rectangular surface. The emission direction of each photon is then set to the unit normal vector of the source plane, which is aligned with the detector plane, thus representing a parallel beam geometry.

Such a rectangular planar source model is also supported in Geant4's `General Particle Source`, where planar shapes (square or rectangle) can be chosen for the position distribution and combined with a *planar* angular type to enforce emission perpendicular to the plane (see `/gps/pos/type Plane`, `/gps/pos/shape Rectangle`, `/gps/ang/type planar`). This configuration is commonly employed in benchmark simulations to model collimated or extended X-ray beams.

9.3 Scattering and Attenuation

Following their emission from the X-ray source, photons traverse space until they encounter matter, where various interaction processes may occur. The subsequent section outlines the fundamental mechanisms of photon interactions with matter. The interaction relevant for medical imaging results in a reduction of radiation intensity, which corresponds to a decreased number of photons reaching the detector. Here, X-ray photons may be fully absorbed by *photoelectric absorption* or undergo either *elastic scattering* (Rayleigh) or *inelastic scattering* (Compton) as they interact with matter.

The attenuation of X-ray photons arises from physical processes that alter their number, direction, or energy as they interact with matter. These interactions occur at the level of individual photons and are highly dependent on the photon energy. This section presents an overview of the primary interaction mechanisms relevant to attenuation like in [17].

For correctness, in the simmulations it is further assumed that the X-ray photons are propagating through vacuum before entering and after exiting the phantom. Usually the tissues are surrounded by air, which has a negligible effect on the photon transport.

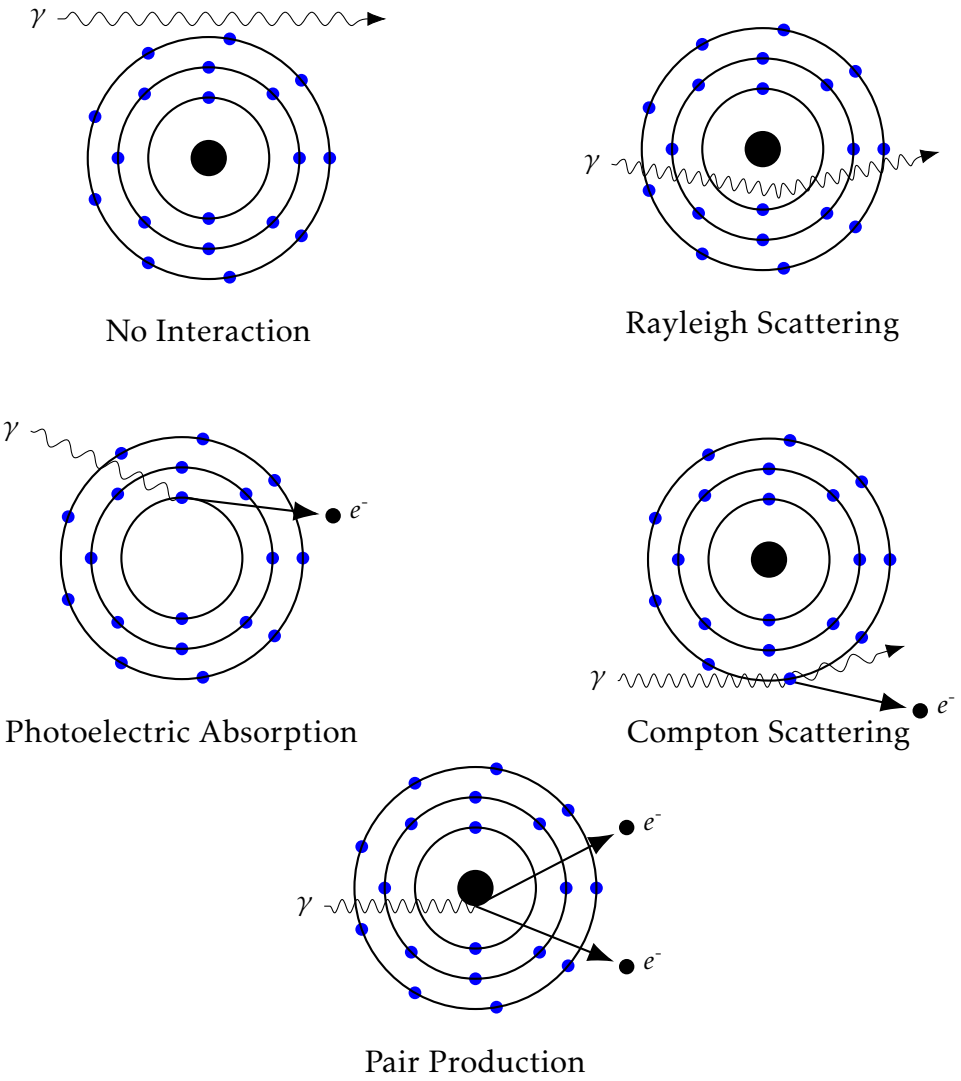


FIGURE 9.6: Principles from photon interaction with matter similar to [17, Chap. 7]

9.3.1 Free Path Length

All mentioned interaction processes - the photoelectric effect, Compton scattering and Rayleigh scattering - are probabilistic in nature. The according attenuation coefficients μ characterizes the extend of the beam being reduced by its according effect, when passing through the material, in $\mathrm{cm}^2\,\mathrm{g}^{-1}$. For the simulation of photons, the attenuation coefficients are dependend on the according energy E of the photon and the material at spherical location x of the photon. The coefficients are summarized in Table 9.1.

Interaction Type	**Attenuation Coefficient**
Photoelectric Effect	$\mu_{\mathrm{PE}}(x,E)$
Rayleigh Scattering	$\mu_{\mathrm{RS}}(x,E)$
Compton Scattering	$\mu_{\mathrm{CS}}(x,E)$

Table 9.1: Attenuation coefficients for different photon interaction mechanisms.

The linear attenuation coefficient $\mu(x,E)$ is the sum of the individual attenuation coefficients of the interaction coefficients:

$$\mu(x,E) = \mu_{\mathrm{PE}}(x,E) + \mu_{\mathrm{RS}}(x,E) + \mu_{\mathrm{CS}}(x,E)$$

When an X-ray photon eneters the phantom two main effects are considered:

- **Attenuation:** The photon may be absorbed or Scattered.
- **No Interaction:** The photon may pass through the phantom without any interaction.

The *free path length* t of a photon describes the distance a photon takes to traverse through the phantom before it interacts with matter. Supposing the phantoms location is x and the photon is traveling in the unit direction $\vec{v}$, as in [15] the free path length t follows the distribution given by:

$$t \sim \mu(x,E)\exp\left[-\int_0^t \mu(x+s\cdot\vec{v})\,ds\right]$$

The free path length t is constraint by the maximum distance c ($0 \le t \le c$) to the next exit point of the phantom along the ray with direction $\vec{v}$. After leaving the phantom, we assume the photon is reaching the detector or leaving the simulation domain.

The probability of the photon to leave the phantom unaltered is described by the *escape probability* $\mathcal{P}$ of a photon at a given position A with unit direction $\vec{v}$ and energy E as in [15]:

$$\mathcal{P}(x,\vec{v},E) = \exp\left[-\int_0^c \mu(x+t\vec{v},E)\,dt\right],$$

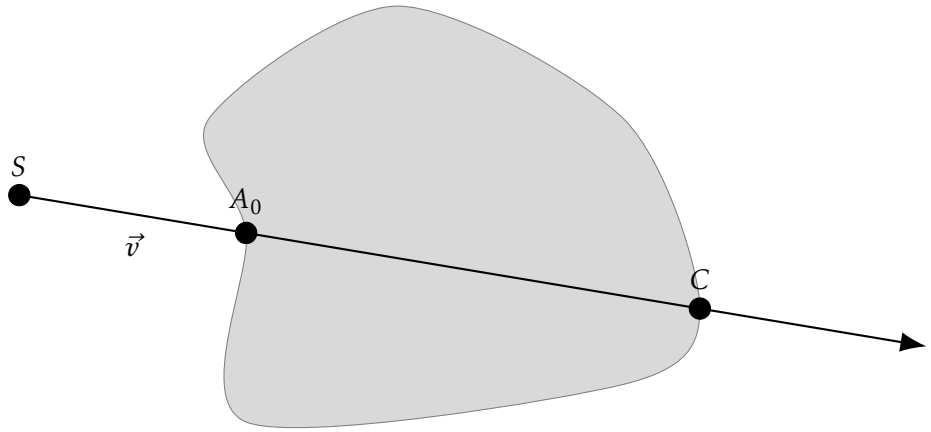


FIGURE 9.7: Illustration of the entry and exit point of the ray of a photon without interaction.

9.3.2 Compton Scattering

Compton scattering is the most dominant interaction mechanism for X-ray photons in tissue [17, Chap. 7]. It occurs when a X-ray photon with considerably higher energy than the binding energy of an outer shell electron collides with this electron. This interaction results in a transfer of energy and momentum. The electron is ejected from the atom, while the incoming photon is scattered at an angle and its energy is reduced.

A portion of the incident photon energy is transferred to the electron, which is referred to as the "recoil electron" or Compton electron. The interaction produces a positive ion, the "recoil electron" and a scattered photon. If the deflection angle of the scattered photon is small, most of the energy is retained by the scattered photon. The deflection angle can vary from 0 to 180 degrees, depending on the energy transfer during the interaction.

For the distribution of the scattering angle θ of the scattered photons in the differential cross section, we apply the formula derived by *Klein and Nishina* in 1928. This equation describes the angular distribution of X-ray photons in Compton scattering, assuming the electron is initially free and stationary. The relation between energy loss and photon deflection is determined from conservation of momentum and energy between the photon and the recoiling electron [11, 13].

$$\frac{d\sigma}{d\Omega}(\theta,E) = \frac{1}{2}r_e^2 \frac{1}{(1+k(1-\cos\theta))^2}\left(1+\cos^2\theta + \frac{k^2(1-\cos\theta)^2}{1+k(1-\cos\theta)}\right), \tag{9.1}$$

where $k = \frac{E}{mc^2} \approx \frac{E}{0.511}$ with mc^2 denoting the rest mass energy of the electron, E the energy of the incident photon and r_e the classical electron radius. Here, σ denotes the scattering cross section, which quantifies the effective area for scattering, and Ω denotes the solid angle element, $d\Omega = \sin\theta\, d\theta\, d\phi$, describing the angular extent in three-dimensional space. The expression $\frac{d\sigma}{d\Omega}$ is thus the *differential cross section*, which represents the probability density of photons being scattered into a particular direction. By integrating over all scattering directions one obtains the total cross

section σ, while normalizing $\frac{d\sigma}{d\Omega}$ yields the probability distribution of the scattering angle θ. This distribution is anisotropic: for low photon energies it is nearly isotropic, whereas for higher energies it becomes increasingly forward peaked.

For sampling from the distribution of the scattering angle we cannot apply the same procedure as for the photon energy in Section 9.2.1, since the Klein-Nishina formula is not a cumulative distribution function (CDF). Instead the method of *rejection sampling* is used.

Rejection sampling is a technique to generate samples from a target distribution by using an envelope distribution that is easy to sample from. The envelope distribution is an upper bound to the original distribution. The basic idea is to sample from this proposal distribution and accept or reject the samples based on a criterion that relates the envelope distribution to the target distribution from Klein-Nishina [19, Chap. 4].

In the context of the distribution of Klein-Nishina, the convergence behavior for $E \to \infty$ and $k \to \infty$ consequently is:

$$f(k,x) \in \mathcal{O}(\frac{1}{k(1-x)}) \tag{9.2}$$

A suitable envelope distribution is of the form

$$h(k,x) = \frac{1}{k(1-x)}$$

with the given conditions

$$\begin{aligned} f(k,1) &= h(k,1) = 2 \\ f(k,-1) &= h(k,-1) = \zeta(k) \\ \zeta(k) &= \frac{2(2k^2+2k+1)}{(2k+1)^3} \end{aligned}$$

By a straightforward calculation, this leads to the following values[21]:

$$\begin{aligned} a(k) &= 2(b(k)-1) \\ b(k) &= \frac{1+\frac{\zeta(k)}{2}}{1-\frac{\zeta(k)}{2}} \end{aligned}$$

Now by using the rejection sampling method, $x = \cos\theta$ is sampled from the distribution h by generating a uniform random variable $r \sim \mathcal{U}(0,1)$ and calculating the value x as follows:

$$x = b(k) - (b(k)-1)\left(\frac{\zeta}{2}\right)^r \tag{9.3}$$

We will accept this value if $\frac{f(k,x)}{h(k,x)} \geq u_1$, where u_1 denotes the random input variable. To derive an exact direction of the scattered photon, we need to further sample an azimuthal angle ϕ, which is done by the second random input variable:

$$\phi = 2\pi u_2 \tag{9.4}$$

Once the scattering angle θ is sampled, the energy of the scattered photon E' can be calculated using the Compton formula:

$$E' = \frac{E}{1 + \frac{E}{m_e c^2}(1 - \cos\theta)} \tag{9.5}$$

9.3.3 Rayleigh Scattering

Rayleigh scattering is a type of elastic scattering that occurs at low X-ray energies [17, Chap. 7]. The incident photon interacts with several electrons that are usually bound in the outer shells of the atom. In this process, the low energy photon is not ejected, but rather the electrons and in turn thw whole atom is set to vibration with respect to the incident photon's wavelength. The vibrating photon transfers its excess energy to an electromagnetic photon with the same wavelength but probably a different direction than the incident photon. The majority of these scattered photons are emitted in a forward direction. This interaction does not result in the ejection of electrons from the atom and no ionization occurs as no energy is converted into kinetic energy.

Although Rayleigh scattering is taking place in the X-ray tube, it can be excluded from the linear attenuation coefficient, as it does not contribute to the attenuation of the X-ray beam in the same way as Compton scattering or photoelectric absorption. As described in [24], the exclusion of Rayleigh scattering from the linear attenuation coefficient is based on the assumption that the characteristic radiation emitted by the target is isotropic, meaning it is emitted uniformly in all directions. As the X-ray is not attenuated by Rayleigh scattering, it is a common assumption to exclude Rayleigh scattering from the attenuation coefficient.

Chapter 10

The Simulation Algorithm

In this chapter, the algorithms developed for X-ray image simulation are presented. To enhance convergence, they integrate aspects of the *Forced Detection* approach introduced in [12].

Unlike standard Monte Carlo simulations, where a random variable determines whether a photon escapes the phantom or undergoes another scattering event, the forced detection approach employed, calculates the probability of escaping the phantom at each interaction point and reflects this distribution in accordingly updated photon intensities for the case of escaping the phantom and the case of scattering. Instead of probabilistically terminating the photon trajectory, the algorithm tracks it through a predefined number of scattering events N. At each interaction, the remaining intensity is updated to reflect both the probability of Compton scattering and the conditional escape probability. This ensures that both potential outcomes — escape or continued scattering — are implicitly accounted for in the evolving photon intensity. As a result, the simulation maintains physical accuracy while achieving a significant reduction in variance.

When a photon eventually escapes the phantom, it contributes to the corresponding detector pixel, weighted by its current intensity, photon energy, and escape probability. Conversely, the distance to the next interaction is sampled based on the probability of not escaping the phantom. Further implementation details are discussed in Section 10.1.

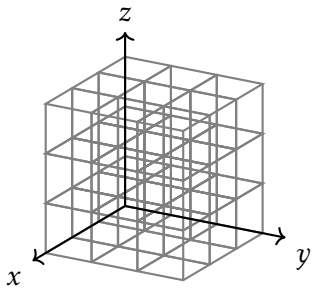
This approach serves as a variance reduction technique by avoiding the all-or-nothing outcomes of conventional photon attenuation sampling, where a photon either escapes and contributes fully to the detector or scatters and may never be detected. Such binary contributions yield high statistical fluctuations. In forced detection, each interaction deterministically splits the photon weight into an escape fraction — proportional to the exponential attenuation along the path to the detector — and a residual fraction that continues via scattering, with free paths sampled from the attenuation law. Thus every photon history contributes to the detector tally, preserving unbiasedness while reducing variance. This enables the simulation to converge more rapidly toward high-resolution images with suppressed noise and well-preserved structural detail.

10.1 Algorithm overview

The algorithm begins by initializing a set of photons, each assigned an initial energy E_0, sampled from the X-ray tube spectrum and an initial direction $\vec{\omega}_0$ sampled

uniformly within a cone centered around the principal beam axis. A total of three random variables are drawn for each photon to determine its energy and direction.

Given a fixed source position A representing the location of the X-ray tube, each photon's initial origin is placed within a voxel grid composed of cubic voxels that define the geometry of the scene. Each voxel encodes an integer value identifying a specific material or tissue type. Furthermore, each photon is initialized with an intensity of $W_0 = 1$, which is iteratively updated throughout the simulation to account for attenuation and scattering processes.



Then an algorithm is applied to traverse the photon through the voxel grid until it reaches the first material which is not air. This is done with a ray traversal algorithm. Once the phantom at a point A_0 is reached, the photon transport is simulated by the physical laws described in Chapter 6.

Here, as in all other scatter points within the phantom, the free *free path length* t_i is sampled from the exponential distribution based on *Beer-Lambert's law* (Eq. 8.1) and the *total attenuation coefficient* $\mu(A_i, E_i)$ accordingly. A_i consequently denotes the current position and E_i the current energy. First, the *escape probability* depending on the current position A_i and the unit direction $\vec{\omega}_i$ and the energy E_i is computed. The escape probability in Equation 10.1 is the probability of the photon escaping the phantom at the current position A_i without being further scattered.

$$p(A_i, \omega_i, E_i) = \exp\left(- \int_{\overrightarrow{A_i C_i}} \mu(x, E_i) dx \right) \tag{10.1}$$

C_i consequently denotes the exit point of the phantom in the direction of the photon $\vec{\omega}_i$. The escape probability is used to determine the intensity of the photon without the $(i+1)$-th scatter event.

Using one random variable u_{3i+1}, the free path length is sampled. Here, both cases are being considered:

1. **Photon escapes the phantom without further scattering:**
   This happens with the portion matching the escape probability $p(A_i, \omega_i, E_i)$. Accordingly
   $$W_{i+1}^{\text{escape}} = W_i \cdot p(A_i, \omega_i, E_i) \tag{10.2}$$
   The photon contributes to the detector signal in direction $\vec{\omega}_i$ with a final intensity of $E_i \cdot W_{i+1}^{\text{escape}}$. In case $i = 0$, this intensity is accounted as priamry intensity.

2. **Photon does not escape the phantom and scatters:**
   This happens with the portion matching the inverse of the escape probability $1 - p(A_i, \omega_i, E_i)$. Accordingly, the free path length is sampled from the exponential distribution by solving Equation 10.3 for t_i:

$$\exp\left(- \int_0^{t_i} \mu(A_i + s \cdot \vec{\omega}_i, E_i) ds \right) = u_{3i+4} \tag{10.3}$$

   Accordingly, the next scatter point is being computed as:

$$A_{i+1} = A_i + t_i \cdot \vec{\omega}_i \tag{10.4}$$

   The photon intensity is then updated with:

$$W_{i+1} = W_i \cdot (1 - p(A_i, \omega_i, E_i)) \cdot \frac{\mu_{\text{CS}}(A_{i+1}, E_i)}{\mu(A_{i+1}, E_i)} \tag{10.5}$$

In the next step, the photon energy and opening angle after the Compton scatter event is sampled with a second random variable u_{3i+2}. A thrid variable is used to apply a azimuthal rotation around the direction of the photon $\vec{\omega}_i$ to sample the new direction $\vec{\omega}_{i+1}$ of the photon after the scatter event.

This process is repeated according to the maximum scatter order N.

TODO: what happens with the leftover intensity? It is forwarded without any further scattering!

10.2 Sub-Algorithms

To model the relevant physical processes in a structured manner, the main simulation is decomposed into several sub-algorithms. This section describes the purpose and implementation of each sub-algorithm in detail.

10.2.1 Photon Generation

For the photon generation step, two fundamental properties are sampled for each photon:

- *Photon energy*, sampled using a single random variable from the X-ray spectrum.
- *Photon direction*, sampled using two random variables uniformly within a cone defined by the beam axis $\vec{d}$ and opening angle α.

Photon Energy Sampling

The photon energy is sampled from the X-ray tube spectrum, which is represented as a discrete set of energy values with corresponding fluence values. The spectrum was generated using *Spekpy* [23, 25], based on an X-ray tube configured with the parameters listed in Table 10.1.

Parameter	Value
Tube voltage	120 kV
Anode material	Tungsten
Filtration	0.4 mm Tin (Sn), 0.1 mm Copper (Cu)
Target angle	12.5°

TABLE 10.1: Parameters used to generate the X-ray spectrum with *Spekpy*.

Photon energies are sampled using inverse transform sampling. The algorithm normalizes the fluence values to obtain a probability density function (PDF), computes the corresponding cumulative distribution function (CDF) and uses a uniformly distributed random variable to select an energy according to the CDF.

Algorithm 3 Photon Energy Sampling from Spectrum

Require: Array of energies $E = [E_1, E_2, \ldots, E_n]$
Require: Corresponding fluence values $\Phi = [\phi_1, \phi_2, \ldots, \phi_n]$
Require: Number of samples N
Require: Random variable $u \sim \mathcal{U}(0,1)$
Ensure: Sampled photon energies $S = [s_1, s_2, \ldots, s_N]$

// Normalize fluence values:

1: $$T \leftarrow \sum_{i=1}^{n} \phi_i$$

2: $$\mathrm{PDF}[i] \leftarrow \frac{\phi_i}{T}$$

// Compute cumulative distribution function:

3: $$\mathrm{CDF}[1] \leftarrow \mathrm{PDF}[1]$$

4: **for** $i = 2$ to n **do**
5: $\quad \mathrm{CDF}[i] \leftarrow \mathrm{CDF}[i-1] + \mathrm{PDF}[i]$
6: **end for**

// Note: $PDF[i] > 0$ in the spectrum, therefore CDF is strictly increasing.

7: Create interpolating function $\mathrm{InverseCDF}(u)$ from $(\mathrm{CDF}[i], E[i])$
8: **return** $\mathrm{InverseCDF}(u)$

Photon Direction Sampling

The direction of each photon is sampled uniformly within a cone defined by the beam axis $\vec{d}$ and opening angle α. This requires two independent random variables $u_1, u_2 \sim \mathcal{U}(0,1)$.

Algorithm 4, adapted from [30], generates a random unit vector $\vec{v}$ uniformly distributed within a cone of half-angle α around the direction $\vec{d}$:

1. **Sampling the polar angle θ**:
   Draw $u_1 \sim \mathcal{U}(0,1)$ and compute

   $$\theta = \arccos\left(1 - u_1(1 - \cos\alpha)\right).$$

   This corresponds to inverse transform sampling such that $\cos\theta$ is uniformly distributed on $[\cos\alpha, 1]$, resulting in uniform sampling over the spherical cap.

2. **Sampling the azimuthal angle ϕ**:
   Draw $u_2 \sim \mathcal{U}(0,1)$ and compute

   $$\phi = 2\pi u_2,$$

   ensuring a uniform distribution around the cone axis.

3. **Constructing the local direction vector**:
   In a local spherical coordinate system with the cone axis aligned along the z-axis, the sampled unit vector is

   $$\vec{v}_{\text{local}} = \begin{pmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{pmatrix}.$$

4. **Rotation into global coordinates**:
   To align the cone axis from the local z-axis to an arbitrary unit vector $\vec{d}$, an orthonormal basis $(\vec{u}, \vec{v}, \vec{d})$ is constructed via the Gram–Schmidt process:

   - Choose a helper vector $\vec{a} = (0,0,1)$ if $|d_3| < 0.999$, otherwise $\vec{a} = (1,0,0)$.
   - Compute $\vec{u} = \frac{\vec{a}\times\vec{d}}{\|\vec{a}\times\vec{d}\|}$.
   - Set $\vec{v} = \vec{d}\times\vec{u}$ to complete a right-handed orthonormal basis.

   Finally, rotate $\vec{v}_{\text{local}}$ into global coordinates via the transformation

   $$\vec{v}_{\text{global}} = (\vec{v}_{\text{local}})_x\vec{u} + (\vec{v}_{\text{local}})_y\vec{v} + (\vec{v}_{\text{local}})_z\vec{d}.$$

Algorithm 4 Uniform Direction Sampling Within a Cone

Require: Cone angle α
Require: Unit beam direction vector $\vec{d} = (d_1, d_2, d_3)$
Require: Random variables $u_1, u_2 \sim \mathcal{U}(0,1)$
Ensure: Sampled direction vector $\vec{v}$ uniformly within cone around $\vec{d}$

// Calculate angles according to samples
$\theta \leftarrow \arccos(1 - u_1(1 - \cos\alpha))$ // Polar angle
$\phi \leftarrow 2\pi u_2$ // Azimuthal angle
// Calculate local direction vector in spherical coordinates

$$\vec{v}_{\text{local}} \leftarrow \begin{pmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{pmatrix}$$

// Orthonormal basis construction (Gram-Schmidt)
if $|d_3| < 0.999$ **then**
  $\vec{a} \leftarrow (0,0,1)$
else
  $\vec{a} \leftarrow (1,0,0)$
end if
$\vec{u} \leftarrow \frac{\vec{a}\times\vec{d}}{\|\vec{a}\times\vec{d}\|}$ // Orthogonal vector
$\vec{v} \leftarrow \vec{d} \times \vec{u}$ // Complete right-handed basis
// Rotate local vector into global coordinates
$\vec{v}_{\text{global}} \leftarrow \vec{u}(\vec{v}_{\text{local}})_x + \vec{v}(\vec{v}_{\text{local}})_y + \vec{d}(\vec{v}_{\text{local}})_z$
return $\vec{v}_{\text{global}}$

10.2.2 Ray Traversal

The ray traversal algorithm is responsible for simulating the propagation of a photon through the voxel grid. It is used to forward the photon until it reaches the first chemical compound, which is not air (or is air). It is also used to simulate the photon transport within the phantom until it reaches the exit point of the phantom.

The Input values of the algorithm are:

- The voxel grid *materialGrid* representing the geometry of the scene.
- The initial position of the photon A.
- The (unit) direction of the photon $\vec{\omega}$.

The algorithm traverses the photon through the voxel grid, voxel by voxel while checking the material of the next voxel and iterrupts in case the next voxel is not air (or is air).

The return values of the algorithm are:

- The coordinates of the final postion: C
- An array of all crossed voxel indices: *crossedVoxels*
- An array of all crossed voxel materials: *crossedMaterials*
- An array of al entry points of each crossed voxel: *entryPoints*

- An array of the distances traveres in each voxel: $distances$
- A boolean mask indicating whether the photon exited the grid: $exitGrid$

The sequence of the algorithm can be summarized as follows:

I. Initialize empty arrays for $crossedVoxels$, $crossedMaterials$, $entryPoints$ and $distances$.

II. Convert the photon position A into voxel coordinates pos.

III. Determine the indices of the next voxel $voxelIdx$ based on the pos and the direction $\vec{\omega}$.

IV. Set $exitGrid$ to false.

V. **If** the $voxelIdx$ of the next voxel is out of bounds, set $exitGrid$ to true and exit the algorithm.

VI. Determine $material$ of the next $voxelIdx$

VII. **If** the $material$ is not air, exit the algorithm.

VIII. **While** $material$ is air:

1. Append $voxelIdx$ to $crossedVoxels$.
2. Append $material$ to $crossedMaterials$.
3. Append pos to $entryPoints$.
4. Determine $maxDist$ to the next voxel boundary.
5. Append $maxDist$ to $distances$.
6. Update pos by adding $\vec{\omega} \cdot maxDist$.
7. Determine $voxelIdx$ of the next voxel based on the updated pos.
8. **If** the $voxelIdx$ is out of bounds, set $exitGrid$ to true and exit the algorithm.
9. Determine $material$ of the next $voxelIdx$.

When the algorithm finishes, the final Position C is set to the last value of $position$.

To use this agorithm for travering the photons through air until they reach the phantom and to traverse the photons through the phantom until they reach the exit point of the phantom, the algorithm is called with the boolean value $throughAir$. In case $throughAir = true$, the algorithm will traverse the photons through air until they reach the first material which is not air. In case $throughAir = false$, the algorithm will traverse the photons through the phantom until they reach the exit point of the phantom.

Therefore we define a short helper function in Algorithm 5 beforehand, which generates the specific expression depending on the boolean value of $throughAir$.

Algorithm 5 Ray Traversal Helper Function

Require: Boolean *throughAir*
Require: *material*
Ensure: Boolean value
1: **if** *throughAir* **then**
2: **return** *material* = air
3: **else**
4: **return** *material* ≠ air
5: **end if**

Algorithm 6 implements the process of ray traversing through the voxel grid in detail.

Algorithm 6 Ray Traversal Algorithm

Require: Boolean $throughAir$
Require: Material grid $materialGrid \in \mathbb{Z}^{X \times Y \times Z}$, voxel size $s \in \mathbb{R}^+$
Require: Initial pos $A \in \mathbb{R}^3$, direction $\vec{\omega} \in \mathbb{R}^3$
Ensure: Final pos $C \in \mathbb{R}^3$
Ensure: Arrays $crossedVoxels, crossedMaterials, entryPoints, distances$
Ensure: Boolean $exitGrid$ indicating whether the photon exited the grid

$pos \leftarrow O/s$
$exitGrid \leftarrow$ false
$currentVoxelIdx \leftarrow \lfloor pos \rfloor$
$negDir \leftarrow \vec{\omega} < 0$
$onB \leftarrow (pos = \lfloor currentVoxelIdx \rfloor)$
$nextVoxelIdx \leftarrow currentVoxelIdx$
$nextVoxelIdx[negDir \wedge onB] \leftarrow nextVoxelIdx[negDir \wedge onB] - 1$
if any$(nextVoxelIdx < 0)$ or any$(nextVoxelIdx \geq$ shape$(materialGrid))$ **then**
    $exitGrid \leftarrow$ true
    $C \leftarrow pos$
    return $C, crossedVoxels, crossedMaterials, entryPoints, distances, exitGrid$
end if
$material \leftarrow materialGrid[nextVoxelIdx]$
while Algorithm 5(throughAir, material) **do**
    $crossedVoxels$.append$(nextVoxelIdx)$
    $crossedMaterials$.append$(material)$
    $entryPoints$.append(pos)
    $fracPos \leftarrow pos - \lfloor pos \rfloor$
    $maxDist \leftarrow [\infty, \infty, \infty]$
    $negDir \leftarrow \vec{\omega} < 0$
    $posDir \leftarrow \vec{\omega} > 0$
    $onB \leftarrow (pos = \lfloor currentVoxelIdx \rfloor)$
    $maxDist[negDir \wedge onB] \leftarrow -1/\vec{\omega}[negDir \wedge onB]$
    $maxDist[negDir \wedge \neg onB] \leftarrow -fracPos[negDir \wedge \neg onB]/\vec{\omega}[negDir \wedge \neg onB]$
    $maxDist[posDir] \leftarrow (1 - fracPos[posDir])/\vec{\omega}[posDir]$
    $distance =$ min$(maxDist)$
    $distances$.append$(maxDist)$
    $pos \leftarrow pos + \vec{\omega} \cdot distance$
    $currentVoxelIdx \leftarrow \lfloor pos \rfloor$
    $nextVoxelIdx \leftarrow currentVoxelIdx$
    $nextVoxelIdx[negDir \wedge onB] \leftarrow nextVoxelIdx[negDir \wedge onB] - 1$
    if any$(nextVoxelIdx < 0)$ or any$(nextVoxelIdx \geq$ shape$(materialGrid))$ **then**
        $exitGrid \leftarrow$ true
        $C \leftarrow pos$
        return $C, crossedVoxels, crossedMaterials, entryPoints, distances, exitGrid$
    end if
    $material \leftarrow materialGrid[nextVoxelIdx]$
end while
$C \leftarrow pos$
return $C, crossedVoxels, crossedMaterials, entryPoints, distances, exitGrid$

10.2.3 Forced Detection

The forced detection algorithm is responsible to apply one iteration of the forced detection process to a photon within the phantom. Consequently, the algorithm accounts:

- The voxel grid *materialGrid* representing the geometry of the scene.
- The voxel grid *totalAttenuationGrid* representing the attenuation coefficients of the materials in the voxel grid *materialGrid*.
- The voxel grid *comptonAttenuationGrid* representing the Compton scattering coefficients of the materials in the voxel grid *materialGrid*.
- The voxel grid *absorptionGrid* representing the absorption coefficients of the materials in the voxel grid *materialGrid*.
- The initial position of the photon A_i.
- The (unit) direction of the photon $\vec{\omega}_i$.
- The initial energy of the photon E_i.
- The initial intensity of the photon W_i.
- A random variable $u \sim \mathcal{U}(0,1)$ to sample the free path length.
- The voxel size s.

The algorithm then applies the ray traversal algorithm (Algorithm 6) to traverse the photon through the phantom until it reaches the exit point of the phantom C_i. Obtaining the exit point C_i, the arrays of *crossedVoxels* and *distances*, now the attenuation coefficients of the materials along the ray can be extracted from the voxel grids for the crossed voxels:

$$\begin{aligned} totalAttenuationCoefficients &= totalAttenuationGrid[crossedVoxels] \\ comptonScatteringCoefficients &= comptonAttenuationGrid[crossedVoxels] \end{aligned}$$

With the following helper algorithm (Algorithm 7), the *escapeProbability* is computed. Further the last distance index k and last interpolation factor f are returned, to solve Equation 10.3 for the free path length t_i, which can then be simply computed by:

$$t_i = \sum_{j=0}^{k-1} distances[j] + f \tag{10.6}$$

Now the next scatter point $A_{i+1} = A_i + t_i \cdot s \cdot \vec{\omega}_i$ can be determined and the new intensities together with the exit point C_i can be computed:

Algorithm 7 Partial Product Sum for Free Path Sampling in Forced Detection

Require: Arrays $distances, totalAttenuationCoefficients \in \mathbb{R}^n$, weight $u \in [0,1]$
Ensure: Last distance index k, last interpolation factor f, $escapeProbability$

$S_{\text{total}} \leftarrow 0$
Initialize array $P[0 \dots n-1]$
for $i \leftarrow 0$ to $n-1$ **do**
    $P[i] \leftarrow distances[i] \cdot totalAttenuationCoefficients[i]$
    $S_{\text{total}} \leftarrow S_{\text{total}} + P[i]$
end for
$T \leftarrow u \cdot S_{\text{total}}$
$S \leftarrow 0$
for $i \leftarrow 0$ to $n-1$ **do**
    if $S + P[i] > T$ **then**
        $f \leftarrow \frac{T-S}{totalAttenuationCoefficients[i]}$
        return (i, f, S_{total})
    end if
    $S \leftarrow S + P[i]$
end for
return $(n, 1.0, S_{\text{total}})$ // u = 1.0 or exact fit

$$voxelIdx = \lfloor A_{i+1}/s \rfloor$$

$$W_{i+1} = W_i \cdot (1 - escapeProbability) \cdot \frac{comptonAttenuationGrid[voxelIdx]}{totalAttenuationCoefficients[voxelIdx]}$$

$$W_{i+1}^{\text{escape}} = W_i \cdot escapeProbability$$

And accordingly the forced detection algorithm returns the following values:

- The new position of the photon A_{i+1}.
- The new intensity of the photon after the scatter event W_{i+1}.
- The intensity of the photon for the case of escaping the phantom W_{i+1}^{escape}.
- The exit point of the phantom C_i.
- The Compton scattering coefficient μ_{CS} at the scatter point A_{i+1}.

Algorithm 8 Forced Detection Algorithm

Require: Material grid $materialGrid \in \mathbb{Z}^{X\times Y\times Z}$
Require: Total attenuation grid $totalAttenuationGrid \in \mathbb{R}^{X\times Y\times Z}$
Require: Compton scattering grid $comptonAttenuationGrid \in \mathbb{R}^{X\times Y\times Z}$
Require: Absorption grid $absorptionGrid \in \mathbb{R}^{X\times Y\times Z}$
Require: Initial position $A_i \in \mathbb{R}^3$
Require: Unit direction $\vec{\omega}_i \in \mathbb{R}^3$
Require: Initial energy $E_i \in \mathbb{R}^+$
Require: Initial intensity $W_i \in \mathbb{R}^+$
Require: Random variable $u \sim \mathcal{U}(0,1)$
Require: Voxel size $s \in \mathbb{R}^+$
Ensure: New position $A_{i+1} \in \mathbb{R}^3$
Ensure: New intensity $W_{i+1} \in \mathbb{R}^+$
Ensure: Escape intensity $W_{i+1}^{\text{escape}} \in \mathbb{R}^+$
Ensure: Exit point $C_i \in \mathbb{R}^3$
Ensure: Compton Attenuation coefficient μ_{CS}

$C_i, crossedVoxels, crossedMaterials, entryPoints, distances, exitGrid \leftarrow$ Algorithm 6$(throughAir = true, materialGrid, s, A_i)$
if exitGrid **then**
    return $(C_i, W_i, 0, C_i)$ // Photon escaped the phantom
end if
$totalAttenuationCoefficients \leftarrow totalAttenuationGrid[crossedVoxels]$
$comptonScatteringCoefficients \leftarrow comptonAttenuationGrid[crossedVoxels]$
$absorptionCoefficients \leftarrow absorptionGrid[crossedVoxels]$
$k, f, escapeProbability \leftarrow$ Algorithm 7$(distances, totalAttenuationCoefficients, u)$
$escapeProbability \leftarrow \frac{escapeProbability}{\sum_{j=0}^{k-1} distances[j]\cdot totalAttenuationCoefficients[j]}$
$t_i \leftarrow \sum_{j=0}^{k-1} distances[j] + f$
$A_{i+1} \leftarrow A_i + t_i \cdot s \cdot \vec{\omega}_i$
$voxelIdx \leftarrow \lfloor A_{i+1}/s \rfloor$
$\mu_{\text{CS}} \leftarrow comptonScatteringCoefficients[voxelIdx]$
$W_{i+1} \leftarrow W_i \cdot (1 - escapeProbability) \cdot \frac{\mu_{\text{CS}}}{totalAttenuationCoefficients[voxelIdx]}$
$W_{i+1}^{\text{escape}} \leftarrow W_i \cdot escapeProbability$
return $(A_{i+1}, W_{i+1}, W_{i+1}^{\text{escape}}, C_i, \mu_{\text{CS}})$

10.2.4 Photon Exit Point Determination

To determine the exit point of the photon in the world after exiting the phantom, a more efficient algorithm than the ray traversal algorithm from Section 10.2.2 is used can be applied. This algorithm yields the exit point of the phantom which is used to determin the detector pixel the photon contributes to.

The algorithm is initialized with the following parameters:

- The voxel grid shape (N_x, N_y, N_z) of the *materialGrid* representing the geometry of the scene.
- The initial position of the photon C^{phantom}.
- The (unit) direction of the photon $\vec{\omega}$.

- The voxel size s.

The algorithm then computes the point where the photon exits the voxel grid efficiently and returns the exit point C^{grid} and the according Coordinates $C^{\text{gridCoords}}$. The algorithm is implemented as follows:

Algorithm 9 Compute Exit Point of Ray from Voxel Grid

Require: Ray origin $C^{\text{phantom}} \in \mathbb{R}^3$, direction $\vec{\omega} \in \mathbb{R}^3$
Require: Grid shape (N_x, N_y, N_z), voxel size $s \in \mathbb{R}^+$
Ensure: Exit point C^{grid}, $C^{\text{gridCoords}}$

$C^{\text{phantomCoords}} \leftarrow C^{\text{phantom}} / s$ // Convert to voxel coordinates
$x_{\min} \leftarrow 0$, $x_{\max} \leftarrow N_x$
$y_{\min} \leftarrow 0$, $y_{\max} \leftarrow N_y$
$z_{\min} \leftarrow 0$, $z_{\max} \leftarrow N_z$
for axis in {x, y, z} **do**
    $o \leftarrow C^{\text{phantomCoords}}_{\text{axis}}$
    $d \leftarrow \vec{\omega}_{\text{axis}}$
    if $d > 0$ **then**
        $t^{(\text{axis})} \leftarrow \frac{x_{\max} - o}{d}$
    else if $d < 0$ **then**
        $t^{(\text{axis})} \leftarrow \frac{x_{\min} - o}{d}$
    else if $d = 0$ **then**
        $t^{(\text{axis})} \leftarrow -\infty$
    end if
end for
$t_{\text{exit}} \leftarrow \min(t^{(x)}, t^{(y)}, t^{(z)})$ // Exit time
$C^{\text{gridCoords}} \leftarrow C^{\text{phantomCoords}} + t_{\text{exit}} \cdot \vec{\omega}$
$C^{\text{grid}} \leftarrow C^{\text{gridCoords}} \cdot s \cdot \vec{\omega}$
return C^{grid}, $C^{\text{gridCoords}}$

10.2.5 Compton Scattering

The Compton scattering algorithm is responsible for simulating the physical process for the event of Compton scattering. Based on the Klein-Nishina formula (Equation 9.1) and the rejection sampling method from Section 9.3.2, the algorithm samples the scatter angle and computes the new photon energy and direction after the scattering event.

The algorithm initializes with the following parameters:

- The initial energy of the photon E_i.
- The (unit) direction of the photon $\vec{\omega}_i$.
- Two random variables $u_1, u_2 \sim \mathcal{U}(0,1)$ to sample the new photon energy and direction.

The algorithm applies the Klein-Nishina procedure to compute the scatter angle and therefore utilizes the electron rest energy E_{rest} and follow this procedure:

1. Initialize

$$k = E_i / mc^2, \qquad \zeta(k) = \frac{2(2k^2+2k+1)}{(2k+1)^3}, \qquad b(k) = \frac{1+\frac{\zeta}{2}}{1-\frac{\zeta}{2}}, \qquad a(k) = 2(b-1).$$

2. Rejection Sampling loop:
   2.1. Generate random number $r \sim \mathcal{U}(0,1)$.
   2.2. Calculate

$$x = b - (b+1)\left(\frac{\zeta}{2}\right)^r$$
$$f(k,x) = \frac{1+x^2+\frac{k^2(1-x)^2}{1+k(1-x)}}{(1+k(1-x))^2}$$
$$h(k,x) = \frac{a}{b-x}$$

   2.3. **If** $u_1 \leq \frac{f(k,x)}{h(k,x)}$ **then**: $\theta = \arccos(x)$

3. Calculate photon energy with Equation 9.5 and direction using the azimuthal angle ϕ from Equation 9.4.

With the random variable u_2 an azimuthal angle ϕ is sampled to determine the new direction.

$$\phi = 2\pi u_2 \tag{10.7}$$

Assuming the vector $\vec{u}$ is an orthogonal unit vector $\vec{u} \perp \vec{\omega}_i$ and $\vec{v} = \vec{u} \times \vec{\omega}_i$, then the new direction $\vec{\omega}_{i+1}$ is as follows:

$$\vec{\omega}_{i+1} = \sin(\theta)cos(\phi) \cdot u + \sin(\theta)sin(\phi) \cdot v + \cos(\theta) \cdot \vec{\omega}_i \tag{10.8}$$

This leads to the following return values of the algorithm:

- The new photon energy E_{i+1} after the Compton scattering event.
- The new (unit) direction $\vec{\omega}_{i+1}$ of the photon after the Compton scattering event.

In the pseudoalgorithm is more explicitly describing this process in detail in Algorithm 10 by implementing omitted tweaks and details.

Algorithm 10 Compton Scattering Algorithm

Require: Initial photon energy E_i, direction $\vec{\omega}_i$
Require: Electron rest energy E_{rest}
Require: Random variables $u_1, u_2 \sim \mathcal{U}(0,1)$
Ensure: New photon energy E_{i+1}, direction $\vec{\omega}_{i+1}$

// Scatter angle sampling
$k \leftarrow E_i / E_{\text{rest}}$
$\varepsilon_0 \leftarrow 1/(2k+1)$
repeat
  Generate $r \sim \mathcal{U}(0,1)$
  if $r < 0.5$ **then**
    $\varepsilon \leftarrow \varepsilon_0 + (1-\varepsilon_0) \cdot 2r$
  else
    $\varepsilon \leftarrow \varepsilon_0 + (1-\varepsilon_0) \cdot 2(1-r)$
  end if
  $\cos\theta \leftarrow 1 + \frac{1}{k}\left(1 - \frac{1}{\varepsilon}\right)$
  if $|\cos\theta| \leq 1$ **then**
    $\sin^2\theta \leftarrow 1 - \cos^2\theta$
    if $u_1 \leq \frac{1}{2}\left(\varepsilon + \frac{1}{\varepsilon} - \sin^2\theta\right)$ **then**
      $\theta \leftarrow \arccos(\cos\theta)$
      break
    end if
  end if
until accepted
// New photon energy calculation
$E_{i+1} \leftarrow \frac{E_i}{1+k(1-\cos\theta)}$
// Orthonormal basis construction
$\phi \leftarrow 2\pi u_2$
if $|(\vec{\omega}_i)_z| < 0.999$ **then**
  $\vec{a} \leftarrow (0,0,1)$
else
  $\vec{a} \leftarrow (1,0,0)$
end if
$\vec{u} \leftarrow \frac{\vec{a} \times \vec{\omega}_i}{\|\vec{a} \times \vec{\omega}_i\|}$
$\vec{v} \leftarrow \vec{\omega}_i \times \vec{u}$
// New direction calculation
$\vec{\omega}_{i+1} \leftarrow \sin\theta\cos\phi \cdot \vec{u} + \sin\theta\sin\phi \cdot \vec{v} + \cos\theta \cdot \vec{\omega}_i$
return $E_{i+1}, \vec{\omega}_{i+1}$

10.3 Algorithm Composition

The main algorithm is composing the sub-algorithms from Section 10.2.

The algorithm is initialized with the following parameters:

- The voxel grid *materialGrid* representing the geometry of the scene.
- The voxel grid *totalAttenuationGrid* representing the total attenuation coefficients of the materials in the voxel grid *materialGrid*.

- The voxel grid *comptonAttenuationGrid* representing the Compton scattering coefficients of the materials in the voxel grid *materialGrid*.
- The voxel grid *absorptionGrid* representing the absorption coefficients of the materials in the voxel grid *materialGrid*.
- The source position S of the X-ray tube.
- The number of scatter events N to simulate.
- The opening angle α of the X-ray beam.
- The voxel size s of the voxel grid.
- A sequence of random variables $u \sim \mathcal{U}(0,1)^{3(N+1)}$ to sample the photon energies, directions and free path lengths.

Thus, the photon generation algorithm is called to generate the photon energies and directions.

Then the photon is initialized with the an initial energy E_0, an initial direction $\vec{\omega}_0$ and an initial intensity $W_0 = 1$ by applying Algorithm 3 and Algorithm 4. Based on the initial position $A_0 = S$, direction $\vec{\omega}$, the voxel grid and voxel size s, the ray traversal algorithm (Algorithm 6) is applied to traverse the photon through the voxel grid until it reaches the first material which is not air. The exit point of the phantom is denoted as C_0.

The loop over the scatter events:

Later N iterations of the forced detection algorithm (Algorithm 8) are applied together with one random variable to simulate the photon transport through the phantom. In each iteration, the photon position and intensity are updated. With Algorithm 9, the exit point C_i^{grid} of the photon is computed and captured together with the relevant intensity $E_i \cdot W_{i+1}^{\text{escape}}$.

Then the Compton scatter event is simulated, taking into account the photon energy E_i, the direction $\vec{\omega}_i$ and the Compton scattering coefficient μ_{CS} at A_{i+1}. Together with two random variables, the new photon energy E_{i+1} with an according direction $\vec{\omega}_{i+1}$ is sampled.

Algorithm 11 Main Simulation Algorithm Composition

Require: Material grid *materialGrid*
Require: Total attenuation grid *totalAttenuationGrid*
Require: Compton attenuation grid *comptonAttenuationGrid*
Require: Absorption grid *absorptionGrid*
Require: Source position S
Require: Number of scatter events N
Require: Beam opening angle α
Ensure: Detector contributions (primary and scatter signals)
Require: Voxel size s
Require: Random variables $u \sim \mathcal{U}(0,1)^{3(N+1)}$

// Photon generation
$E_0 \leftarrow$ Algorithm 3(spectrum, u_1)
$\vec{\omega}_0 \leftarrow$ Algorithm 4(α, beam axis, u_2, u_3)
$W_0 \leftarrow 1$
// Traverse photon through air to phantom
$A_0, \ldots \leftarrow$ Algorithm 6(throughAir=true, *materialGrid*, s, S, $\vec{\omega}_0$)
for $i = 0$ to $N-1$ **do**
// Forced detection step
$A_{i+1}, W_{i+1}, W_i^{\text{escape}}, C_i^{\text{phantom}}, \mu_{\text{CS}} \leftarrow$ Algorithm 8(*materialGrid*, *totalAttenuationGrid*, *comptonAttenuationGrid*, *absorptionGrid*, A_i, $\vec{\omega}_i$, E_i, W_i, u_{3i+1}, s)
// Record escaped photon contribution
if $W_i^{\text{escape}} > 0$ **then**
$C_i^{\text{world}}, \ldots \leftarrow$ Algorithm 9(C_i^{phantom}, $\vec{\omega}_i$, grid shape, s)
Add $E \cdot W_i^{\text{escape}}$ to detector pixel at C_i^{world}
end if
// Compton scatter step
$E_{i+1}, \vec{\omega}_{i+1} \leftarrow$ Algorithm 10(E, $\vec{\omega}_i$, μ_{CS}, u_{3i+2}, u_{3i+3})
end for
// Handle leftover intensity
$C_N^{\text{world}}, \ldots \leftarrow$ Algorithm 9(A_N, $\vec{\omega}_N$, grid shape, s)
Add $E \cdot W$ to detector pixel intensity at C_N^{world}

Bibliography

[1] Takuya Akiba et al. "Optuna: A next-generation hyperparameter optimization framework". In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2623–2631.

[2] Vladimir I Bogachev. *Measure theory*. Springer, 2007.

[3] Russel E Caflisch. "Monte carlo and quasi-monte carlo methods". In: *Acta numerica* 7 (1998), pp. 1–49.

[4] Xiangning Chen et al. "Symbolic discovery of optimization algorithms". In: *Advances in neural information processing systems* 36 (2023), pp. 49205–49233.

[5] Geant4 Collaboration. *Geant4: A Simulation Toolkit*. Version 11.3.2. 2025. URL: https://geant4.web.cern.ch/.

[6] George Cybenko. "Approximation by superpositions of a sigmoidal function". In: *Mathematics of control, signals and systems* 2.4 (1989), pp. 303–314.

[7] Bastien Doignies. "Echantillonnage différentiable et simulations Monte Carlo". PhD thesis. Université Claude Bernard-Lyon I, 2024.

[8] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[9] John H Halton. "On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals". In: *Numerische Mathematik* 2.1 (1960), pp. 84–90.

[10] Michael Hardy. "Combinatorics of partial derivatives". In: *arXiv preprint math/0601149* (2006).

[11] J Hubbell. "Photon cross sections, attenuation coefficients and energy absorption coefficients". In: *National Bureau of Standards Report NSRDS-NBS29, Washington DC* (1969).

[12] H.W.A.M. de Jong, E.T.P. Slijpen, and F.J. Beekman. "Acceleration of Monte Carlo SPECT simulation using convolution-based forced detection". In: *IEEE Transactions on Nuclear Science* 48.1 (2001), pp. 58–64. DOI: 10.1109/23.910833.

[13] Oskar Klein and Yoshio Nishina. "The scattering of light by free electrons according to Dirac's new relativistic dynamics". In: *Nature* 122.3072 (1928), pp. 398–399.

[14] Gunther Leobacher and Friedrich Pillichshammer. *Introduction to quasi-Monte Carlo integration and applications*. Springer, 2014.

[15] Guiyuan Lin, Shiwo Deng, and Xiaoqun Wang. "An efficient quasi-Monte Carlo method with forced fixed detection for photon scatter simulation in CT". In: *PLOS ONE* 18 (Aug. 2023), e0290266. DOI: 10.1371/journal.pone.0290266.

[16] Guiyuan Lin et al. "Scatter correction based on quasi-Monte Carlo for CT reconstruction". In: *arXiv preprint arXiv:2501.05039* (2025).

[17] *Medical Imaging Systems : An Introductory Guide*. eng. Image Processing, Computer Vision, Pattern Recognition, and Graphics; 11111. Cham, 2018. Chap. 7,8. ISBN: 9783319965208.

[18] Siddhartha Mishra and T Konstantin Rusch. "Enhancing accuracy of deep learning algorithms by training with low-discrepancy sequences". In: *SIAM Journal on Numerical Analysis* 59.3 (2021), pp. 1811–1834.

[19] Thomas Müller-Gronbach, Erich Novak, and Klaus Ritter. *Monte Carlo-Algorithmen*. Springer-Verlag, 2012.

[20] Art B Owen. "Multidimensional variation for quasi-Monte Carlo". In: *International Conference on Statistics in honour of Professor Kai-Tai Fang's 65th birthday*. World Scientific. 2005, pp. 49–74.

[21] Emin N Özmutlu. "Sampling of angular distribution in Compton scattering". In: *International journal of radiation applications and instrumentation. Part A. Applied radiation and isotopes* 43.6 (1992), pp. 713–715.

[22] Adam Paszke et al. "Pytorch: An imperative style, high-performance deep learning library". In: *Advances in neural information processing systems* 32 (2019).

[23] Gavin Poludniowski. *SpekPy: Python package for simulating X-ray spectra*. Version 2.0.13. 8.08.2024. URL: https://bitbucket.org/caxtus/book/src/master/.

[24] Gavin Poludniowski, Artur Omar, and Pedro Andreo. *Calculating X-ray Tube Spectra: Analytical and Monte Carlo Approaches*. CRC Press, 2022.

[25] Gavin Poludniowski et al. "SpekPy v2. 0—a software toolkit for modeling x-ray tube spectra". In: *Medical Physics* 48.7 (2021), pp. 3630–3637.

[26] SciPy Developers. *Quasi-Monte Carlo Sampling Tools: `scipy.stats.qmc`*. https://docs.scipy.org/doc/scipy/reference/stats.qmc.html. Accessed: 2025-08-18. 2025.

[27] A. Sisniega et al. "Automatic Monte-Carlo Based Scatter Correction For X-ray cone-beam CT using general purpose graphic processing units (GP-GPU): A feasibility study". In: *2011 IEEE Nuclear Science Symposium Conference Record*. 2011, pp. 3705–3709. DOI: 10.1109/NSSMIC.2011.6153699.

[28] Michael Spivak. *Calculus on manifolds: a modern approach to classical theorems of advanced calculus*. CRC press, 2018.

[29] Jörg Steidel et al. "Dose reduction potential in diagnostic single energy CT through patient-specific prefilters and a wider range of tube voltages". In: *Medical physics* 49.1 (2022), pp. 93–106.

[30] Murugesan Venkatapathi et al. "An O (n) algorithm for generating uniform random vectors in n-dimensional cones". In: *arXiv preprint arXiv:2101.00936* (2021).

[31] Pauli Virtanen et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python". In: *Nature methods* 17.3 (2020), pp. 261–272.

[32] Markus Zahrnhofer. *Einführung in Quasi-Monte Carlo Verfahren*. 2007.